Frontispiece

TRAINING MANUAL

Coding With //Screen-Readers

CONFIDENTIAL — For Internal Use Only

Copyright © 2026 Michael Ryan Hunsaker, M.Ed., Ph.D. Coding With Screen-Readers is licensed under a Creative Commons Attribution-NoNCommerical-NoDerivatives 4.0 International: https://creativecommons.org/licenses/by-nc-nd/4.0/ All rights reserved. NO AI TRAINING: Without in any way limiting the author's rights under copyright, any use of this publication to "train" generative artificial intelligence (AI) technologies to generate text is expressly prohibited.

Contents

Teacher's Guide: Introduction to Computer Science with Python 1
Unit 0: Setting Up Your Accessible Python Development Environment 43
Unit 1: Foundations of Computing and Getting Started with Python 50
Unit 2: Writing Simple Programs and Numeric Computations 56
Unit 3: Control Flow — Decisions and Loops 63
Unit 4: Data Structures — Lists and Dictionaries 69
Unit 5: User Input and `while` Loops 72
Unit 6: Functions 75
Unit 7: Classes and Object-Oriented Programming 80
Unit 8: Files and Exceptions 84
Unit 9: Testing and Data Analysis 88
Unit 10: Project-Based Learning — Building Text-Based Applications 93
Unit 11: Capstone Project — Accessible Personal Data Manager 99

Dedication

To the countless individuals who have been told they lack the capability to code, yet remain determined to learn and create through accessible coding practices, this book is dedicated.

Epigraph

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect

— Tim Berners-Lee, Inventor of the World Wide Web

Teacher's Guide: Introduction to Computer Science with Python

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Foundational Principles
2. Understanding Your Students
3. Classroom Setup and Technology Configuration
4. Universal Spoken Code Conventions
5. Unit 0 — Setup and Environment
6. Unit 1 — Foundations and Getting Started
7. Unit 2 — Numeric Computations and I/O
8. Unit 3 — Control Flow, Decisions, and Loops
9. Unit 4 — Data Structures: Lists and Dictionaries
10. Unit 5 — User Input and While Loops
11. Unit 6 — Functions
12. Unit 7 — Classes and Object-Oriented Programming
13. Unit 8 — Files and Exceptions
14. Unit 9 — Testing and Data Analysis
15. Unit 10 — Project-Based Learning
16. Unit 11 — Capstone Project
17. Assessment Strategies
18. Supporting Diverse Needs Within the Visually Impaired Population
19. Teaching AI Tools Responsibly
20. Alternative and Supplementary Resources
21. Quick Reference Sheets

---

1. Philosophy and Foundational Principles

Why Python for Blind Students? Python is widely recognized as one of the most accessible first programming languages for visually impaired learners for several concrete reasons:

- **Text-based I/O is the default.** Unlike languages that push beginners toward GUI frameworks or graphical plotting, Python's most natural outputs are plain text to the terminal — fully accessible to any screen reader.
- **Indentation enforces structure.** While indentation introduces a learning challenge (addressed in detail below), it also means that code read aloud by a screen reader carries structural meaning in the form of announced indent levels — something curly-brace languages do not provide.

- **Error messages are descriptive.** Python's tracebacks include the file name, line number, error type, and a message. A screen reader user can navigate directly to the problem without visual scanning.
- **The standard library is enormous.** Python ships with modules for file I/O, JSON, math, random numbers, CSV parsing, unit testing, and more — reducing the complexity of dependency management in a learning context.
- **The REPL is an accessible sandbox.** Python's interactive interpreter provides immediate spoken feedback for any expression, enabling rapid experimentation without the overhead of creating and running files.

Core Pedagogical Commitments **Commit to ears-first instruction.** Every demonstration, every example, every debugging session should be conducted as if the teacher is also using a screen reader. Ask yourself before every lesson: "If I could not see the screen, how would I know what was happening right now?" If the answer is unclear, redesign the demonstration.

Commit to explicit navigation. Sighted students can glance at code and immediately locate structures. Blind students must navigate deliberately using keyboard shortcuts and screen reader commands. Teach navigation skills as primary content, not auxiliary support.

Commit to verbal precision. Vague instructions like "look at the next line" or "you can see here that…" create invisible barriers. Replace all visual language with navigation language: "Move down two lines," "Jump to line 14 with Ctrl+G," "Navigate to the end of the file with Ctrl+End."

Commit to error-positive culture. For a sighted student, an error is a red underline they can ignore or address at leisure. For a screen reader user, an unaddressed error is noise that clutters the output of every subsequent navigation. Treat errors as immediate priorities and build the habit of reading and resolving them before moving on.

Commit to redundancy. Provide concepts in multiple formats — spoken explanation, written pseudocode, physical analogy, and code. Blind students cannot fall back on visual re-reading; redundant representation builds retention.

---

2. Understanding Your Students

The Heterogeneity of Visual Impairment "Visually impaired" encompasses a wide spectrum. Your students may include:

Profile	Implications for Teaching
Congenitally blind (blind from birth)	May have no visual spatial metaphors. Analogies must be tactile, auditory, or conceptual. May be faster with screen readers from years of experience.
Late-onset blindness (lost vision after sighted years)	May retain strong visual mental models. Can use these as bridges but should not be required to. May still be learning screen reader skills.
Low vision (functional but impaired vision)	May use a combination of magnification and screen reading. May be able to see some screen content but cannot rely on fine detail. Do not assume screen-only or vision-only.
Color blindness	Primarily relevant for syntax highlighting. Ensure themes are not relied upon for meaning.
Progressive conditions	Vision may change during the course. Build habits that remain useful at any level of vision.

Never assume a student's profile without asking. Some students are private about their diagnoses; frame questions around what tools they use and what works best, not what their condition is.

Screen Reader Literacy Varies Some students arrive with years of fluent screen reader experience. Others are relatively new to their screen reader and may still be developing efficiency. Assess screen reader fluency separately from programming aptitude — a student who struggles to navigate a file is not struggling with Python; they need more time with their tool.

Suggest that students spend time with their screen reader in non-programming contexts (browsing documentation, reading formatted text) before or alongside the first programming units.

Braille Display Users Some students use a refreshable braille display in addition to or instead of audio output. Key considerations:

- Braille displays have limited width (typically 40 or 80 cells). Long lines of code may need to be panned across. Encourage keeping line length under 79 characters (PEP 8 standard) for this reason.
- Code rendered in braille uses Grade 1 (uncontracted) braille because contractions would misrepresent code characters. Confirm this setting is active.
- Navigation with a braille display typically involves routing cursor buttons — physical buttons above each braille cell that move the cur-

sor to that position. Students using these may navigate differently than audio-only users.
- Braille display users often prefer lower verbosity audio output since they read braille silently. Do not require audio announcements as the only feedback mechanism.

Students Who Are Also Deaf-Blind Students who are deaf-blind rely exclusively on braille for all information. Audio feedback from the screen reader is not available. Considerations:

- All instructional communication must be via tactile means, text-based communication, or tactile sign language interpreters.
- Braille output of code is the primary interface. Pay special attention to line length and symbol representation.
- Terminal output must be navigable via braille display — avoid programs that produce output faster than a braille display can be read.
- Refer to specialists in deaf-blind education for specific accommodation strategies beyond the scope of this guide.

---

3. Classroom Setup and Technology Configuration

Physical Environment

- Ensure the student's workstation does not have excessive background noise that interferes with screen reader audio. Headphones are often preferable in a shared classroom.
- Position the workstation so the teacher can sit beside (not in front of) the student, allowing them to listen to the screen reader output together.
- If a classroom assistant or orientation and mobility specialist is present, coordinate roles in advance. The programming teacher leads on programming content; the VI specialist leads on accessibility tool configuration.

VS Code Configuration Checklist Work through this checklist with the student on day one:

Screen Reader

- ☐ Screen Reader Optimized Mode ON (Ctrl+Shift+P -> Toggle Screen Reader Optimized)
- ☐ Accessibility help panel reviewed (Shift+Alt+F1)

Editor Settings (Ctrl+,)

- ☐ editor.lineNumbers = "on"

- ☐ editor.renderWhitespace = “all” or “boundary”
- ☐ editor.wordWrap = “on” (for long lines)
- ☐ workbench.reduceMotion = “on” (reduces distracting animations)
- ☐ editor.tabSize = 4
- ☐ editor.insertSpaces = true (spaces not tabs for Python)

Screen Reader Indentation (in screen reader settings, not VS Code)

- ☐ NVDA: Document Formatting -> Indentation -> “Say all” or “Beeps”
- ☐ JAWS: Configure speech for VS Code environment
- ☐ VoiceOver: Verbosity -> Text -> announce indentation
- ☐ Narrator: No built-in indent announcement; use Beep plugin or extension

Extensions Installed

- ☐ Python (Microsoft)
- ☐ Pylance
- ☐ Python Indent (for consistent auto-indentation)
- ☐ GitLens or Git Graph (for version control visibility)
- ☐ Error Lens (inline error annotations — screen-reader-readable)

Keyboard Shortcuts Practiced

- ☐ Ctrl+G — Go to line
- ☐ F8 — Next error
- ☐ Shift+F8 — Previous error
- ☐ Ctrl+Shift+M — Problems panel
- ☐ Shift+Alt+F2 — Accessible View
- ☐ Ctrl+Shift+P — Command Palette
- ☐ Ctrl+‘ — Toggle terminal
- ☐ Ctrl+Shift+E — Explorer
- ☐ Ctrl+Shift+O — Go to symbol (function/class navigation)
- ☐ Alt+F12 — Peek Definition
- ☐ F12 — Go to Definition
- ☐ Shift+F12 — Find all References

Terminal Configuration

- **Windows Terminal** (preferred over legacy cmd.exe on Windows) has better accessibility with NVDA and JAWS.
- **NVDA + Windows Terminal:** Enable “Use UI Automation” in NVDA settings for better terminal reading.
- **VoiceOver + macOS Terminal:** Works well by default. Use `VO+Shift+Down` to interact with terminal content.
- **Orca + Linux terminal:** Orca reads terminal output well. Use `Orca+T` (flat review) to re-read recent output.

- Configure the terminal font to a monospace font at a size comfortable for any residual vision.

```
# Create environment with a consistent name the student will always type
python3 -m venv .venv

# Activation --- save this as a text file the student can open and copy from
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate

# Verify with a spoken command
python3 --version
pip list
```

Python Virtual Environment Setup for Accessibility Consider creating a startup script (`start.bat` on Windows, `start.sh` on macOS/Linux) that activates the environment and opens VS Code. Students can run one command to get their environment ready, reducing setup cognitive load.

---

4. Universal Spoken Code Conventions

Establish this vocabulary on the first day and reinforce it consistently. When a student is reading code aloud (for debugging, peer review, or just narrating their work), they should use these terms without hesitation.

Symbol Pronunciation Table

Written Symbol	Spoken Form	Notes
`=`	"gets"	Assignment only
`==`	"equals equals" or "is equal to"	Comparison
`!=`	"not equal to" or "bang equals"	
`>=`	"greater than or equal to"	
`<=`	"less than or equal to"	
`>`	"greater than"	
`<`	"less than"	
`+=`	"plus equals" or "add and assign"	
`-=`	"minus equals"	

Written Symbol	Spoken Form	Notes
`*=`	“times equals”	
`**`	“to the power of” or “double star”	
`//`	“floor divide” or “integer divide”	
`%`	“modulo” or “remainder”	
`()`	“open paren … close paren”	Always say both
`[]`	“open bracket … close bracket”	
`{}`	“open brace … close brace”	
`:`	“colon”	End of if/for/def/class line
`"text"`	“quote text quote”	
`'text'`	“single-quote text single-quote”	
`f"..."`	“f-string”	Mention the f prefix
`{var}` inside f-string	“open brace var close brace”	
`#`	“hash”	Followed by comment text
`\n`	“backslash n” or “newline”	
`\t`	“backslash t” or “tab”	
`_`	“underscore”	In names: “my underscore name”
`__init__`	“dunder init”	Double-underscore methods
`__str__`	“dunder str”	
`.`	“dot”	In `math.sqrt`, say “math dot sqrt”
`->`	“arrow”	Type hints
`*args`	“star args”	
`**kwargs`	“double-star kwargs”	
`None`	“None” (capital N)	Distinct from the word “none”
`True` / `False`	“True” / “False” (capital)	
`import`	“import”	

Written Symbol	Spoken Form	Notes
`from x import y`	"from x import y"	Read as English sentence

Reading a Line of Code Aloud Teach students to read full lines this way, left to right, then explain:

```
total_price = unit_price * quantity + tax
```

"total underscore price gets unit underscore price times quantity plus tax"

Then explain: *"This line calculates the total price by multiplying the unit price by the quantity, then adding tax, and stores the result in total_price."*

The two-pass approach — first read the symbols, then explain the meaning — builds both syntax recognition and semantic understanding.

---

5. Unit 0 — Setup and Environment

Lesson 0.1: Installing Python and VS Code

Before This Lesson Survey the student's current technology setup:

- What screen reader do they use, and what version?
- Do they use a braille display?
- What operating system?
- Have they used a terminal before?
- Do they have administrator access to their computer?

Confirm administrator access before scheduling this lesson. Installation without admin rights will fail.

Instructional Cues Walk through every installer step verbally as the student performs it. Do not say "click the next button" — say "Tab until you hear 'Next button,' then press Enter." Treat this as teaching screen reader navigation, not just software installation.

Critical checkpoint — PATH configuration: After installation, before anything else: `python3 --version`. If this fails, stop and address the PATH issue before proceeding. A student who continues without Python on PATH will encounter mysterious failures throughout the course.

Non-Obvious Accessibility Strategies

- **Windows Defender SmartScreen:** On Windows, running a downloaded `.exe` from Python.org may trigger a SmartScreen dialog. This

dialog can be navigated with Tab and Enter but is not always announced immediately by NVDA. Tell the student what to expect before they run the installer.

- **macOS Gatekeeper:** Downloaded packages may require right-click -> Open to bypass security. On a screen reader, right-click is VO+Shift+M (VoiceOver) or App Key (NVDA with keyboard access).
- **Linux package managers as an alternative:** For Linux students, `sudo apt install python3 python3-pip python3-venv` is often more reliable than the Python.org download and is fully scriptable.
- **Portable Python for school computers:** Some schools restrict software installation. Python.org offers portable/embeddable distributions. Replit.com is a web-based Python IDE that requires no installation and is reasonably accessible.

Alternative Installation Methods

Method	Accessibility	Best For
Python.org installer	High — standard dialog flow	Most installations
Chocolatey (Windows)	High — command-line, screen-reader friendly	Windows power users
Homebrew (macOS)	High — command-line	macOS users comfortable with terminal
apt/dnf/pacman (Linux)	High — command-line	Linux users
Replit.com	Medium — web-based, some accessibility gaps	No local install possible
Thonny IDE	Medium — simpler UI, bundled Python	Beginners who struggle with VS Code

On Thonny: Thonny is a Python IDE specifically designed for beginners. It has a simpler interface than VS Code and is sometimes more accessible for students who are new to both screen readers and programming. The trade-off is fewer advanced accessibility features long-term. Consider Thonny for the first 2–3 lessons to reduce setup burden, then migrate to VS Code.

Thought Experiments

1. *"Why do you think the Python installer has a checkbox called 'Add Python to PATH'? What is a PATH, and what would happen if Python weren't on it? Think about it like a phonebook — your operating system looks up programs by name. If Python isn't in the phonebook, what happens when you type `python3`?"*

2. *"We're installing two things: Python and VS Code. What role does each one play? If you had only Python and no VS Code, could you still write programs? How? What would you lose?"*
3. *"Virtual environments keep each project's dependencies separate. Imagine two school projects — one requiring a specific edition of a textbook that contradicts the edition another class uses. What would happen if you tried to use both editions at the same time? How does a virtual environment solve the programming equivalent of this problem?"*
4. *"Why do we verify every installation immediately after completing it? What could go wrong between downloading something and confirming it works? Name three things that could silently fail."*

Lesson 0.2: Configuring Your Screen Reader for VS Code

Instructional Cues Before showing any configuration screens, do an unguided exploration: "Open VS Code, navigate around with only your keyboard and screen reader, and come back in ten minutes. Tell me what you found, what worked, what was confusing." This builds genuine familiarity and identifies that student's specific challenges.

Map each configuration step to a problem the student already identified. This makes the configuration feel purposeful rather than arbitrary.

Screen Reader Specific Guidance **NVDA users:**

- Enable "Use UI Automation to access VS Code" in NVDA preferences (Tools -> Manage Plugins, or via NVDA+N -> Preferences -> Settings -> Advanced -> Use UI Automation where available). This dramatically improves reading accuracy in VS Code.
- Set "Report dynamic content changes" to "All" to hear error highlights and IntelliSense popups.
- Configure Browse Mode to be OFF by default in VS Code — programming requires forms mode (interactive mode) at all times.
- Speech rate: many experienced NVDA users run at 70–90% of maximum speed. For new students, 40–60% allows better comprehension of code read aloud.

JAWS users:

- JAWS has VS Code-specific scripts maintained by the JAWS community. Check `FreedomScientific.com` and GitHub for the latest VS Code JAWS scripts.
- PlaceMarkers in JAWS can mark frequently visited locations in a file — teach `Ctrl+Win+K` to set and `Ctrl+Win+J` to jump.

- JAWS Verbosity settings: set "Announce line numbers" to ON for code files.

VoiceOver (macOS) users:

- VoiceOver interacts with VS Code via the macOS Accessibility API. Ensure VS Code is not run with elevated permissions (which can break accessibility API access).
- Use `VO+Shift+Down` to enter interactive elements (the editor, terminal), then navigate normally.
- Enable "Speak text under mouse" in VoiceOver Utility -> Verbosity if the student uses residual vision with a mouse.
- VoiceOver's "item chooser" (VO+I) can provide a quick list of interactive elements in the current window.

Windows Narrator users:

- Narrator is less feature-rich than NVDA or JAWS for programming. It works but may require more keystrokes.
- Narrator scan mode must be OFF in VS Code for normal keyboard input to work. Toggle with Caps Lock+Space.
- Consider recommending the free NVDA screen reader as an upgrade for students using Narrator, as NVDA's VS Code support is significantly better.

Orca (Linux) users:

- Orca works well with VS Code on Linux via the AT-SPI2 accessibility interface.
- Use `Orca+H` for "say all" from cursor position.
- Ensure the `at-spi2-core` package is installed: `sudo apt install at-spi2-core`.
- Orca's "flat review" mode (`Orca+KP_Enter` or `Orca+Insert+KP_Enter`) is useful for reading lines that are not focus-accessible.

Indentation Configuration — Step by Step This is the most important configuration in the entire course. Walk through it during the first session and verify it is working before writing any Python.

Test procedure:

1. Create a temporary file `test_indent.py`.
2. Write:

```
if True:
    print("indented")
print("not indented")
```

3. Navigate line by line with Down arrow.

4. The screen reader should announce the indent level on line 2 before reading its content.

5. On line 3, the indent should return to zero.

If announcement is not happening, troubleshoot screen reader settings before proceeding.

Thought Experiments

1. *"A sighted programmer can look at a Python file and immediately understand its structure from indentation — like an outline. How does your screen reader translate that same 2D visual structure into a 1D audio stream? What information might be lost in that translation?"*

2. *"VS Code has hundreds of keyboard shortcuts. No sighted programmer uses all of them — they use the 15 or 20 that matter for their workflow. If you could only keep 10 shortcuts, which 10 would make the biggest difference for writing Python? Why?"*

3. *"The Accessible View (Shift+Alt+F2) converts complex visual elements into flat text. What kind of information in VS Code might require this conversion? Can you think of a scenario where the 'visual' presentation of something would be unusable without translation?"*

Lesson 0.3: Setting Up Your VS Code Workspace

Alternative Workspace Tools If VS Code proves too complex initially, consider these alternatives as temporary or permanent solutions:

Tool	Platform	Accessibility Notes
IDLE (bundled with Python)	All	Very simple UI. Limited features but highly keyboard accessible. Good for first programs.
Thonny	All	Beginner-focused. Has a "simple mode." Some screen reader users find it more approachable.
Mu Editor	All	Designed for education. Simpler than VS Code. Python 3 only.
Jupyter Notebook (terminal)	All	Text-based notebook. Accessible for structured exploration.
Braille Note Apex/Touch	Device	For students using these notetakers, Python can be run via their built-in terminals.

Tool	Platform	Accessibility Notes
Replit	Web	No install required. Accessibility varies; test before recommending.
NVDA Remote + VS Code	Windows	Student uses their own screen reader on their own machine while teacher controls a shared session.

Thought Experiments

1. *"Programmers organize their code into folders by project. What happens if every programmer at a company kept all their files in one shared folder with no organization? How does this analogy apply to organizing your own Python projects?"*

2. *"VS Code Extensions add features to the editor the same way apps add features to a phone. If you could design one VS Code extension specifically for blind Python programmers that doesn't exist yet, what would it do?"*

Lesson 0.4: Python Environments and Terminal Usage

Terminal Proficiency as a Prerequisite Before writing any Python, ensure the student can fluently:

- Navigate to a directory with `cd`
- List files with `ls` (macOS/Linux) or `dir` (Windows)
- Create a directory with `mkdir`
- Print the current directory with `pwd` (macOS/Linux) or `cd` with no arguments (Windows)
- Use Tab completion for filenames
- Use Up/Down arrow for command history
- Use `Ctrl+C` to stop a running program

If these are not automatic, spend additional time on terminal basics. Terminal fluency is foundational to everything else.

Alternative Terminals for Accessibility

Terminal	Platform	Screen Reader Compatibility
Windows Terminal	Windows	Excellent with NVDA/JAWS (use UI Automation mode)
cmd.exe	Windows	Good; legacy but very reliable
PowerShell	Windows	Good with NVDA; some JAWS quirks

Terminal	Platform	Screen Reader Compatibility
macOS Terminal	macOS	Excellent with VoiceOver
iTerm2	macOS	Good; more features, slightly more complex
GNOME Terminal	Linux	Good with Orca
Konsole	Linux	Good with Orca
Terminator	Linux	Good; split-pane support

Thought Experiments

1. *"Virtual environments isolate each project's dependencies. Why would this matter for a professional programmer with 20 active projects? What could go wrong without them?"*

2. *"When you type `python3 hello.py` and press Enter, what is the operating system doing? What is Python doing? Walk through it step by step as if explaining to someone who has never used a computer."*

3. *"`pip install requests` downloads and installs code written by other people. What are the risks of running code written by strangers? What does PyPI (the Python Package Index) do to reduce these risks?"*

---

6. Unit 1 — Foundations and Getting Started

Lesson 1.1: Welcome to Computer Science

Instructional Cues Open with **the "Human Computer" activity** — this is the most valuable exercise in the entire unit for blind students. The student gives verbal, step-by-step instructions for a simple physical task. The teacher follows them literally, making no inferences. This demonstrates the core challenge of programming: computers are perfectly literal and do not fill in gaps.

Suggest tasks that connect to the student's daily experience:

- Instructions for making tea or coffee
- Instructions for navigating from the classroom door to their desk
- Instructions for finding a contact in their phone using a screen reader

After the activity, debrief: *"Where did your instructions assume I could see? Where did they assume I already knew something? A computer makes neither assumption."*

Connecting Computing to Accessibility Technology Begin with what the student already knows:

- Their screen reader is software. It runs on hardware. It was written by programmers. It converts visual information (text rendering) into audio output.
- Ask: "What are the inputs to your screen reader? What are the outputs? What would happen if a website used images of text instead of real text — how would that change what your screen reader can do?"
- Connect to ARIA attributes, alt text, and semantic HTML as programming concepts that directly affect the student's daily experience.

Thought Experiments

1. *"Computer science is described as the study of problem-solving, not just programming. What non-programming problems have you solved using computational thinking — breaking a problem down into steps, looking for patterns, and abstracting the details? Describe one example from daily life."*

2. *"Your screen reader converts text to audio in real time. What 'algorithm' do you think it uses? What decisions does it make — when to pause, how to handle punctuation, what to do with symbols? What would a flowchart of those decisions look like?"*

3. *"Early computers could only run one program at a time. Modern computers run hundreds simultaneously. Without changing the physical hardware, what concept makes this possible? (Hint: the answer is software — specifically, an operating system.)"*

4. *"Software has often been described as 'making the hardware do something useful.' But hardware without software is just metal and silicon. In what sense is software more important than hardware? In what sense is hardware more important?"*

5. *"Accessibility technology like screen readers, braille displays, and audio description didn't exist when computers were first invented. Who decided these should be built? What does this history tell you about who gets to shape what technology does?"*

Additional Resources for This Lesson

- **"The Computer Boys Take Over" by Nathan Ensmenger** — accessible history of how programming became a profession.
- **"Technically Wrong" by Sara Wachter-Boettcher** — explores how biases in tech affect users, including disabled users. Good for older students.

- **American Foundation for the Blind (AFB) CareerConnect** — profiles of blind professionals in STEM fields, including software engineers. Good for motivation and role models.
- **"Haben: The Deafblind Woman Who Conquered Harvard Law" by Haben Girma** — autobiography of a deafblind lawyer and disability rights advocate. Haben's work directly involves technology access.
- **Code.org's "How Computers Work" video series** — audio-described versions available. Covers hardware and software basics accessibly.

Lesson 1.2: Your First Python Program

Instructional Cues **Before any code:** Ask the student to predict. "If I write `print("Hello, World!")` and run it, what do you think will appear in the terminal? Where exactly will it appear?" Let them answer, then verify together.

Make the first error deliberate. Remove a closing parenthesis, run the program, and navigate to the error message together. Establish the error-reading protocol:

1. Terminal output appears after the code.
2. Read from the bottom of the error upward — the bottom is the most specific message.
3. Line number tells you where to go. Use `Ctrl+G`.
4. Error type (`SyntaxError`, `NameError`) narrows the category of problem.

The REPL as the first environment. Before creating any files, start with `python3` in the terminal and type `print("Hello, World!")` directly. The immediate audio feedback of the REPL establishes cause and effect more directly than the file -> run -> find output workflow.

Common First Errors and How to Explain Them

Error	Likely Cause	Explanation for Student
`SyntaxError: unexpected EOF while parsing`	Missing closing `"`, `)`, or `]`	"Python reached the end of the file before it expected to. It was waiting for something — probably a closing character."

Error	Likely Cause	Explanation for Student
`NameError: name 'print' is not defined`	Capital P in `Print`	"Python is case-sensitive. `Print` and `print` are completely different names. Only `print` (lowercase) is the built-in function."
`IndentationError`	Extra space before first line	"Python's first rule: top-level code cannot be indented. Every line that is not inside a function or class must start at column 0."
`SyntaxError: invalid syntax` at `:`	Using = instead of == in a comparison	"The assignment operator = doesn't belong inside an if condition. For comparison, you need the double equals ==."

Thought Experiments

1. *"`print()` produces output in the terminal. What would a program be like with NO output at all? Could such a program still do something useful? (Hint: think about programs that read files, send network requests, or modify a database.)"*
2. *"Python is an interpreted language — each line is read and executed one at a time. Other languages are compiled — the entire program is translated before any of it runs. How does this affect the kind of errors you encounter, and when you encounter them?"*
3. *"The `#` symbol starts a comment — text that Python ignores entirely. Why do comments exist? If the computer ignores them, who are they for? Could you write good programs with no comments? Should you?"*
4. *"'Hello, World!' is the traditional first program in almost every programming language. Why do you think this tradition exists? What does this simple program prove about a working environment?"*

Alternative Resources for This Lesson

- **"Invent Your Own Computer Games with Python" by Al Sweigart (free online):** Text-based games that produce only terminal output — perfectly accessible. Chapter 1 is a good companion to this lesson.
- **CS Unplugged — "Parity Magic" activity:** Teaches binary representation and algorithms without a computer. Can precede or follow this lesson.
- **Python REPL via iOS/Android with screen reader:** Several apps provide a Python REPL on mobile. For students whose phone screen

reader is more familiar than their computer setup, this may be a comfortable way to experiment.

Lesson 1.3: Variables and Simple Data Types

Instructional Cues **The labeled box metaphor** works well across modalities — introduce it verbally and reinforce with tactile analogy if helpful. Provide a physical box (a small container) with a label stuck to it and a slip of paper inside representing the value. The box remains; the paper changes. The label stays the same; the contents change with reassignment.

`type()` as a diagnostic reflex: Begin every data type lesson with `type()`. `print(type(42))` -> `int`. `print(type("hello"))` -> `str`. Make this a reflex the student uses whenever they're unsure what kind of data they have. For a student who cannot visually scan code for context clues, `type()` is often the only reliable way to confirm what a variable contains.

The string/number audio confusion: When a screen reader reads `"42"` and `42`, they sound identical. This is a critical accessibility issue. Address it directly:

- *"Your screen reader will read these the same way. The only reliable way to tell them apart is `type()`. Use it. Constantly."*
- Demonstrate the consequence: `"42" + 1` raises `TypeError`. `42 + 1` gives `43`.

Multiple Explanations for Variable Naming Rules Provide all of these; let the student choose what sticks:

- *Rule-based:* Must start with letter or underscore. May contain letters, digits, underscores. May not be a Python keyword.
- *Analogy:* "Variable names are like legal names. They can't start with a number (no one's legal name is '2Fast'), can't contain hyphens (the name Mc-Donald would confuse paperwork), and can't be reserved words like 'True' or 'if' that already mean something official."
- *Mnemonic for invalid starts:* "Numbers first? Never. Hyphens? Horrible. Spaces? Surely not."

Thought Experiments

1. *"If variables are labeled boxes, what happens when you do `x = 5` and then `y = x` and then `x = 10`? What is `y` now? Does reassigning `x` affect `y`? Why or why not?"*

2. *"Python is 'dynamically typed,' meaning a variable can change its type at any time:* `x = 5` *then* `x = 'hello'`*. Many languages forbid this. What bugs could dynamic typing accidentally create? What flexibility does it offer?"*

3. *"The integer* `42` *and the string* `'42'` *look identical when spoken aloud. How many other pairs of values can you think of that might be ambiguous when a screen reader reads them? What coding habits help prevent confusion?"*

4. *"Constants in programming represent values that never change — like* `SPEED_OF_LIGHT` *or* `MAX_USERS`*. Python has no true constants (you can always reassign them). Why do programmers use* `CAPS_SNAKE_CASE` *as a convention for constants even though Python can't enforce them?"*

Alternative Resources

- **CS50 Python (Harvard, edX):** David Malan's accessible video lectures. Closed-captions and transcripts available for all lessons. Covers variables and types in Week 0.
- **Python.org official tutorial:** Plain HTML, navigable by screen reader. Section 3 covers numbers, strings, and basic types.
- **"Python for Everybody" by Dr. Chuck Severance (free online):** Audio-described video lectures available. Chapter 2 covers variables and expressions.
- **Codecademy "Learn Python 3":** Web-based coding environment. Accessibility is partial — code editor is accessible but some UI elements are not. Worth testing with the student's specific setup.

---

7. Unit 2 — Numeric Computations and I/O

Lesson 2.1: The Software Development Process

Instructional Cues This is a conceptual lesson. Resist the urge to rush to code. Spend the full session on design and pseudocode.

The "Five Whys" technique: Before writing pseudocode, ask the student to state the problem in one sentence. Then ask "why?" five times to reach the root of what the program really needs to do. This technique from Lean manufacturing translates well to programming.

Example: "My program calculates a restaurant tip."

- Why? "So the user knows how much to pay."
- Why do they need to calculate it? "The tip amount isn't printed on the bill."
- Why? "The restaurant leaves the amount to the customer's discretion."

- So the real problem is: "Given a bill amount, calculate 15%, 18%, and 20% tip options, plus the total."

This more complete problem statement produces a better program design.

Pseudocode format for screen reader users: Plain `.txt` files work best for pseudocode. Avoid word processors or special formatting tools that add accessibility overhead. The student should be able to read their pseudocode fluently while writing their Python.

Thought Experiments

1. *"In engineering, a bridge is designed many times before a single beam is placed. The cost of redesigning a blueprint is near zero; the cost of redesigning a half-built bridge is enormous. How does this ratio (cheap to redesign early, expensive to redesign later) apply to software?"*

2. *"The design phase involves thinking about edge cases — unusual inputs. For a program that calculates how many buses are needed to transport N students, with each bus holding 40 students, what edge cases exist? (Consider: What if N=0? What if N=40? What if N=41? What if N is negative?)"*

3. *"Some agile software development methodologies say 'working software over comprehensive documentation.' Does this conflict with the value of design? How do you reconcile the two?"*

4. *"Testing is described as part of the development process. But some teams test after development is complete. What is lost by testing at the end rather than continuously throughout? What is gained?"*

Lessons 2.2–2.5: Expressions, Input, Types, Math

Instructional Cues **`input()` always returns a string — make it tangible:**

```
user_input = input("Type the number 5: ")
print(type(user_input))   # <class 'str'>
print(user_input + 1)      # TypeError
print(int(user_input) + 1) # 6
```

Run this sequence and let the student hear both the error and the fix. The `TypeError` message should be read aloud and analyzed before the fix is shown.

Order of operations — the hearing test: Read `10 + 5 * 2` aloud and ask for the answer before Python is consulted. Most students say 30;

Python says 20. This is a memorable demonstration of why precise symbol reading matters. "Your ear parsed it as 'add 10 and 5, then multiply by 2.' Python parses it as '10 plus the result of 5 times 2.'"

`math` module exploration via REPL:

```
import math
dir(math)  # Lists all functions in the module
help(math.sqrt)  # Reads the docstring aloud
```

`dir()` and `help()` are screen-reader-friendly tools for exploring any module. Teach them as first-resort documentation tools.

Thought Experiments

1. *"Type conversion functions like `int()` and `float()` can fail: `int('hello')` raises a `ValueError`. Who is responsible — the programmer who called `int()` on untrusted input, or the user who typed 'hello'? How should a well-designed program handle this?"*

2. *"`math.sqrt(25)` returns `5.0`, not `5`. Why does a function that logically produces a whole number return a float? What would break if it returned an integer instead?"*

3. *"Floating-point numbers in computers are approximations — `0.1 + 0.2` gives `0.30000000000000004`. Describe a real-world application where this imprecision could cause serious problems. What strategies exist to avoid it?"*

4. *"The `//` operator performs floor division: `7 // 2` = `3`. The `%` operator gives the remainder: `7 % 2` = `1`. Together they completely describe integer division. What real-world problem does each solve? Describe a program that would need both."*

Alternative Resources

- **"Automate the Boring Stuff with Python" Chapter 1–3 (free online):** Covers expressions, data types, and input with practical exercises. Plain HTML, accessible.
- **Khan Academy "Introduction to Programming":** Video series with transcripts. Good for visual-thinker students who benefit from hearing concepts explained differently.
- **SoloLearn Python course:** Mobile app with accessible exercises. Works well with iOS VoiceOver and Android TalkBack for students who prefer mobile.
- **"Python Crash Course" by Eric Matthes:** The primary textbook for this curriculum. Chapter 2 aligns directly with this unit.

8. Unit 3 — Control Flow, Decisions, and Loops

Lessons 3.1–3.2: Conditionals

Instructional Cues **Verbal simulation protocol:** Before running any conditional code, have the student perform the simulation verbally:

1. "What is the value of `score`?"
2. "Evaluate the condition: is `85 >= 90`? That's False."
3. "Because the condition is False, the if block is skipped."
4. "Evaluate the next condition: is `85 >= 80`? That's True."
5. "The elif block runs. We print 'Grade: B'."

This 5-step protocol should become automatic before the student runs any code. It builds the mental simulation ability that is the core of debugging.

The assignment-vs-comparison trap:

```
# This is a bug that is SILENT in some languages
if score = 90:   # SyntaxError in Python (good!)
    print("Perfect!")
```

Python raises a `SyntaxError` here, which is actually a safety feature. Other languages (C, JavaScript) allow this and it becomes a silent logic bug. Appreciate Python's strictness here.

Thought Experiments

1. *"If Python didn't have `elif`, only `if` and `else`, could you still write a grading program? Show how. What is the cost of doing it without `elif`?"*
2. *"The `else` clause at the end of an if-elif chain is a catch-all — it runs if nothing else matched. What is the programmer asserting when they write `else: print('Invalid input')`? What assumptions about the universe of possible inputs does this make? When might those assumptions be wrong?"*
3. *"Boolean logic (`and`, `or`, `not`) comes from mathematics (George Boole, 1847). Boolean logic also describes how circuits in a computer work. In what sense is every program you write built on top of millions of tiny `and`, `or`, and `not` operations happening in hardware?"*
4. *"Short-circuit evaluation: in `if A and B`, Python doesn't evaluate `B` if `A` is False (because the result is already False). In `if A or B`, Python doesn't evaluate `B` if `A` is True. When is this behavior a useful feature rather than just an optimization?"*
5. *"Consider a program that decides whether to grant access based on two conditions: correct password AND recognized device. Map this to Python `and` logic. Now consider: unlock if either condition passes (maybe for emergency access). Map this to `or`. What are the security implications of each design?"*

Alternative Resources

- **CS Unplugged "Binary Numbers" activity:** Physical, non-computer activity that illustrates Boolean thinking. Can be adapted for tactile use.
- **"Think Python" by Allen B. Downey (free online):** Chapter 5 covers conditionals with clear explanation and exercises. HTML version is screen-reader-friendly.
- **Parsons Problems (online):** Code ordering exercises — the student arranges given lines into correct order. Does not require typing code from scratch. Available at parsons.problemsolving.org (accessibility varies; test first).

Lessons 3.3–3.4: Lists and For Loops

Instructional Cues **Mental models for lists — choose one per student:**

- *The train:* Each car has a number starting at 0. Car 0 is at the front. You can add cars to the end, remove from anywhere, and swap contents.
- *The playlist:* Songs in order, starting at position 0. You can shuffle, sort, add, or remove tracks.
- *The physical stack of cards:* A sequence of items that can be addressed by position.

The for loop as a bookmark reader: *"Imagine Python has a finger that moves along a list. It starts at position 0. It reads what's there, runs the loop body, moves the finger to position 1, reads, runs the body, and continues until the finger falls off the end."*

Tracing loops audibly — required exercise: For the first five loops students write, require them to verbally trace every iteration before running the code:

- "Iteration 1: `fruit` gets 'apple'. We print 'I like apple.' "
- "Iteration 2: `fruit` gets 'banana'. We print 'I like banana.' "
- etc.

Non-Obvious Accessibility Strategies

- **Printing list contents accessibly:** A bare `print(my_list)` produces `['apple', 'banana', 'cherry']` with brackets and quotes. This is cluttered when read aloud. Teach the clean print pattern early:

```
for i, item in enumerate(my_list):
    print(f"  {i}: {item}")
```

This gives numbered output, one item per line, that is easy to navigate and understand when heard.

- **`len()` as a safety check before index access:** Teach students to always know the length before indexing into a list. This prevents `IndexError`. Make `len(my_list)` the first thing printed when debugging a list problem.
- **Index arithmetic verbalized:** When accessing `my_list[-1]`, say "the last element, which is at index `len(my_list) minus 1`, which Python shortens to `-1`." Connecting the shorthand to the underlying meaning prevents confusion.

Thought Experiments

1. *"Zero-based indexing means the first element is at position 0. This feels unnatural but is standard across almost all programming languages. Why? (Hint: think about memory addresses — the address of the first element, and the address of any element as an offset from there.)"*
2. *"A list allows duplicate values. A set (which you'll learn later) does not. When would duplicates in a list be meaningful data? When would they indicate an error in your program?"*
3. *"A `for` loop iterates over every element in a list. Is there any computation a `for` loop can do that a `while` loop cannot? Is there any computation a `while` loop can do that a `for` loop cannot?"*
4. *"What happens when you iterate over an empty list: `for item in []:`? Does Python crash, print an error, or do nothing? Why is 'do nothing' the correct behavior rather than an error?"*

Lesson 3.5: String Operations

Instructional Cues **Immutability demonstration:**

```
text = "hello"
text.upper()        # Creates a new string --- doesn't modify text
print(text)         # Still "hello"
text = text.upper() # Now reassigned
print(text)         # "HELLO"
```

This demonstration catches a persistent beginner misunderstanding. Ask the student to predict the output of `print(text)` after `text.upper()` before running it.

Method chaining verbalized: `"  hello world  ".strip().title()` — read this as: "Start with the string. Strip whitespace from both ends. Then convert the result to title case."

Thought Experiments

1. *"Strings are immutable — they cannot be changed, only replaced. Lists are mutable — they can be changed in-place. What architectural difference between strings and lists makes one mutable and the other not? (Think about how they're stored in memory.)"*
2. *"`.split()` splits a string into a list of words. `.join()` combines a list of strings into one string. They are inverses. What is the inverse of `.replace()`? Does it have a direct inverse? If not, how would you reverse a replacement?"*
3. *"String formatting with f-strings: `f'Value is {x:.2f}'` formats `x` as a float with 2 decimal places. Why does a programming language need a mini-language for formatting numbers? What would programs look like if we had to format every number manually with division and modulo?"*

Alternative Resources

- **Python String Methods documentation** (docs.python.org/3/library/stdtypes.html#string-methods): Plain HTML table of all string methods. Screen-reader navigable. Essential reference.
- **"Python for Data Analysis" by Wes McKinney:** Chapter on string manipulation provides real-world examples beyond what this curriculum covers. For motivated students.

---

9. Unit 4 — Data Structures: Lists and Dictionaries

(See all lessons above for list material; this section adds dictionary-specific guidance.)

Dictionary Lessons

Instructional Cues **Multiple mental models for dictionaries:**

- *Real dictionary:* You look up a word (key) to find its definition (value). Words are unique; definitions can be anything.
- *Contact list:* Each name (key) maps to a phone number (value). No two contacts have the same name in a well-maintained list.
- *JSON configuration file:* Settings names map to their values. This foreshadows Unit 8.

The `.get()` method vs. direct access:

```
# This crashes on missing key
print(my_dict["nonexistent_key"])   # KeyError

# This returns a default --- never crashes
print(my_dict.get("nonexistent_key", "Not found"))  # "Not found"
```

For a screen reader user who may not immediately catch a crash in terminal output, teaching `.get()` as the default access pattern reduces debugging overhead.

Thought Experiments

1. *"Dictionaries use keys to retrieve values in O(1) time — essentially instant regardless of dictionary size. Lists use indices, also O(1). But searching a list for a specific value (not by index) takes O(n) time — proportional to list length. When does this difference matter in practice?"*

2. *"Your dictionary cannot have duplicate keys — adding a second value for an existing key replaces the first. What real-world data problems does this property solve? What problems does it create?"*

3. *"JSON (JavaScript Object Notation) is essentially Python dictionaries written in a slightly different format. What does it mean that a data format invented for JavaScript has become the universal language of data exchange? What does this tell you about the value of common standards?"*

4. *"A 'nested dictionary' contains other dictionaries as values. Model a university course catalog as a nested dictionary. What are the outer keys? What are the inner keys? What would `catalog['CS']['101']['instructor']` represent?"*

---

10. Unit 5 — User Input and While Loops

Thought Experiments

1. *"A while loop with a flag variable and a while loop with `break` can often accomplish the same thing. Compare these two equivalent programs and explain when you would prefer each:*

```
# Version A: flag
active = True
while active:
    cmd = input("> ")
    if cmd == "quit": active = False
    else: print("You said:", cmd)

# Version B: break
while True:
    cmd = input("> ")
    if cmd == "quit": break
    print("You said:", cmd)
```

Which is clearer? Which is easier to extend if you add more exit conditions?"

2. *"An infinite loop that waits for user input is the core pattern of every interactive program — email clients, video games, operating systems. Describe the 'event loop' of your screen reader in terms of while loops and if statements. What is it waiting for? What does it do when it gets an event?"*

3. *"The `continue` statement skips the rest of the current iteration. An alternative is to wrap the loop body in an `if` statement. When is `continue` cleaner? When does the `if` approach produce clearer code?"*

---

11. Unit 6 — Functions

Thought Experiments (in addition to existing guide)

1. *"A function is described as a 'black box' — you give it inputs and get outputs without knowing the internal details. What are the benefits of this abstraction for the programmer who writes the function? For the programmer who uses it? What are the limits of this metaphor?"*

2. *"Default parameter values: `def greet(name='World')`. This allows the function to be called with or without an argument. How does this enable backward compatibility — adding a new feature without breaking existing code that calls the function?"*

3. *"Functions should 'do one thing.' What does 'one thing' mean? Is `calculate_and_print_tax()` one thing or two? If you separate it into `calculate_tax()` and `print_tax()`, what becomes possible that wasn't before?"*

4. _"Recursive functions call themselves. `factorial(n) = n _ factorial(n-1)`. What is the base case and why is it required? What goes wrong if a recursive function has no base case?"*

5. _"A function is a way to name a piece of code. Why does naming matter in programming? What is the cognitive difference between `x = a _ b + c - d`and`x = calculate_net_profit(revenue, cost, overhead, taxes)`?"*

Alternative Resources

- **"Structure and Interpretation of Computer Programs" (SICP) — MIT OpenCourseWare:** Conceptually deep treatment of functions as first-class objects. Lectures have transcripts. For advanced students.
- **Exercism.io Python Track:** Mentor-guided exercises with community feedback. Web platform with reasonable accessibility. Students can complete exercises at their own pace.

- **Python Tutor (pythontutor.com):** Step-by-step execution with variable state display. The text-based "state panel" below the code is screen-reader accessible and shows variable values at each step.

---

12. Unit 7 — Classes and Object-Oriented Programming

Extended Instructional Cues **Staged introduction to OOP — do not rush to syntax:**

Stage 1 (no code): Discuss the concept. What is an object? What does "bundling data and behavior" mean? Use tangible examples the student knows.

Stage 2 (pseudocode): Write a class in plain English before Python syntax:

```
Class: BankAccount
  Has: owner_name, balance, account_number
  Can do: deposit(amount), withdraw(amount), get_balance()
```

Stage 3 (Python skeleton): Write the class structure with empty methods and docstrings.

Stage 4 (full implementation): Fill in method bodies.

This staging prevents syntax confusion from obscuring the conceptual learning.

`__str__` is mandatory for accessibility: Every class a student creates must define `__str__`. Without it, `print(my_object)` produces `<__main__.BankAccount object at 0x7f...>` — meaningless when heard. The requirement to define `__str__` also reinforces thinking about what the object's most important information is.

```
def __str__(self):
    return f"BankAccount(owner={self.owner_name}, balance=${self.balance:.2f})"
```

Thought Experiments

1. *"Encapsulation means hiding internal details. A car's engine is encapsulated under the hood — you don't need to know how fuel injection works to drive. What are the analogous 'hidden internals' in a BankAccount class? What should be private? What should be public? Why?"*
2. *"Inheritance creates 'is-a' relationships. Composition creates 'has-a' relationships. A `SavingsAccount` is-a `BankAccount`. A `Bank` has-a list of `BankAccount` objects. Design a simple banking system using both relationships. What would go wrong if you tried to use only inheritance?"*
3. *"Method overriding allows a subclass to replace a parent's method. When is this the right design? When should you call `super().method()` inside the override vs. replace the method entirely?"*

4. *"Python allows multiple inheritance (inheriting from two parent classes). Most languages do not permit this. What problems does multiple inheritance create? What problems does it solve? Would you use it?"*

5. *"Object-oriented programming was supposed to make code more reusable. Has it delivered on this promise? What kinds of code reuse are well-served by OOP? What kinds aren't?"*

Alternative Resources

- **"Object-Oriented Programming in Python" by Michael Goldwasser and David Letscher:** Textbook specifically pairing Python and OOP concepts. More formal treatment for students who want depth.
- **"Fluent Python" by Luciano Ramalho:** Advanced but deeply accessible to screen readers. Covers Python's data model including `__str__`, `__repr__`, and special methods. For motivated students.
- **Real Python "Object-Oriented Programming in Python" tutorial (realpython.com/python3-object-oriented-programming/):** Long but thorough. Plain HTML.

---

13. Unit 8 — Files and Exceptions

Extended Instructional Cues **Files as persistence — make it tangible:** Run a program that stores a number in a variable, quits, and starts again — the variable is gone. Then run the same program storing the number in a file — it persists. Ask: "What changed? What is different about a file vs. a variable?" Let the student reason through it before explaining.

The `with` statement — demonstrate the alternative:

```
# Without 'with' --- dangerous
f = open("data.txt", "r")
contents = f.read()
# If an error occurs here, f.close() is never called
f.close()

# With 'with' --- safe
with open("data.txt", "r") as f:
    contents = f.read()
# f is closed automatically even if an error occurs
```

Exception types — teach by demonstration, not memorization: Create a "crash collection" — a series of small programs that each produce a specific exception. Let the student run each one, read the error, and discuss why it occurred before seeing the solution. The goal is pattern recognition, not memorization.

Thought Experiments

1. *"Files are persistent storage — they survive when your program exits. Variables are temporary — they're gone when the program ends. What determines which data should be stored in files vs. kept only in memory? What are the tradeoffs?"*

2. *"The `with` statement uses Python's 'context manager' protocol. This protocol says: 'when this block starts, do X; when it ends (even if it ends with an error), do Y.' What other situations in programming require guaranteed cleanup? What happens in real systems when cleanup is skipped?"*

3. *"Exceptions propagate up the call stack — if function A calls B which calls C, and C raises an exception, it moves up to B, then A, until something catches it. If nothing catches it, the program crashes. Design the exception handling strategy for a program that reads a config file, parses it, and then connects to a database. Where should exceptions be caught?"*

4. *"JSON stores data as text. Binary formats (like `pickle` in Python) store data in a compact binary form. JSON is human-readable; binary is not. JSON is language-independent; pickle is Python-only. For each of these use cases, which format would you choose: (a) a config file the user might edit by hand, (b) a large dataset processed only by your own Python code, (c) data shared between a Python backend and a JavaScript frontend?"*

---

14. Unit 9 — Testing and Data Analysis

Extended Instructional Cues **Tests as documentation — make this concrete:** Show two identical functions — one with tests that describe the expected behavior, one without. Ask: "Which function is easier to understand without reading the implementation?" The tests are a specification in executable form.

Test-driven development (TDD) as a preview: For motivated students, introduce TDD: write the test before the implementation. This inverts the usual workflow:

1. Write a test that fails (because the function doesn't exist yet).
2. Write the minimum implementation to make the test pass.
3. Refactor.

This discipline is genuinely useful for beginners because it forces them to think about what a function should do before worrying about how.

Thought Experiments

1. *"Unit testing tests one function in isolation. Integration testing tests multiple components working together. System testing tests the entire program. Why have three levels rather than just one? What does each level catch that the others might miss?"*
2. *"A test can only prove that a program behaves correctly for the inputs tested. It cannot prove correctness for all possible inputs. What does this mean for the value of testing? How do you decide which inputs to test?"*
3. *"Random numbers make testing difficult — if the output changes every run, how do you write an assertion? Describe a strategy for testing a function that uses `random.randint()`. (Hint: think about seeding.)"*
4. *"CSV data from the real world is often 'dirty' — missing values, extra commas, inconsistent formatting, encoding issues. What does a robust CSV parser need to handle? What assumptions would a naive parser make that would cause it to fail on real-world data?"*

---

15. Unit 10 — Project-Based Learning

Extended Instructional Cues **Design document requirements:** Require a written design document before any code is written. For a screen reader user, a plain text file works best. The document should include:

```
Project Name: [title]
One-sentence purpose: [what problem does this solve?]
Target user: [who will use this?]

Features (ranked by priority):
1. [most important feature]
2. ...

Data I will store: [what information needs to persist?]
Data structures: [which Python structures will hold this?]

Functions I will write:
- [function name](inputs) -> output: [brief description]
- ...

Main loop pseudocode:
[plain English description of program flow]

Edge cases I have considered:
- [what happens if the user enters nothing?]
- [what happens if the file doesn't exist?]
- ...
```

Iterative development enforcement: Require demonstrating a working version at the end of every session, no matter how minimal. "Working" means runnable and producing correct output for at least one input. A student who cannot demonstrate working code at session end needs to reduce scope, not work harder overnight.

Thought Experiments

1. *"A program designed for sighted users might use color to indicate status (red for error, green for success), icons to represent actions, and layout to group related information. How would you replicate each of these design patterns in a screen-reader-accessible terminal program?"*

2. *"Your contact manager has a search function. How many different search behaviors could be useful? Search by exact name match, partial match, phone prefix, email domain, notes keywords? What should the program do when there are 0 results? 1 result? Multiple results?"*

3. *"Error messages should be actionable — they should tell the user not just what went wrong, but what to do next. Rewrite these unhelpful messages to be actionable: 'Error: invalid input', 'File not found', 'Operation failed.'"*

---

16. Unit 11 — Capstone Project

Assessment Guidance

Process Checkpoints

Checkpoint	Timing	What to Evaluate
Written design document	Before any code	Is the scope realistic? Are features prioritized? Are data structures appropriate?
Data model working	End of week 1	Can items be created, stored in memory, and printed?
Core CRUD functional	End of week 2	Add, view, edit, delete all work?
Persistence working	End of week 3	Data survives program restart?
Error handling complete	End of week 4	Program handles all invalid inputs without crashing?
Accessibility review	End of week 4	Is all output meaningful when read aloud?
Final demo	End	Full demonstration with screen reader active

The Screen Reader Demo Rubric The final demonstration should be performed entirely with the screen reader active. Evaluate each:

Criterion	Excellent	Satisfactory	Needs Work
Menu navigation	Student navigates menu by listening; confirms choice before acting	Student navigates but occasionally needs visual cues	Student needs visual confirmation of menu state
Input prompts	All prompts clearly describe expected format and type	Most prompts are descriptive	Some prompts are vague or missing
Confirmation messages	Every action produces a clear spoken confirmation	Most actions confirmed	Some actions succeed silently
Error messages	All error messages are specific and actionable	Most errors give useful messages	Some errors produce generic or missing messages
Data persistence	Student demonstrates save and load by quitting and restarting	Save/load works but student doesn't demonstrate live	Save or load not fully functional
Code explanation	Student can explain any line of their code when asked	Student can explain most of their code	Student cannot explain sections of their code

---

17. Assessment Strategies

Accessible Assessment Design Traditional programming assessments (written tests, timed coding) create unnecessary barriers for blind students. Use alternatives that assess the same competencies:

Think-aloud problems: Give the student a short piece of code and ask them to read it aloud and explain what it does, step by step. This assesses reading comprehension and debugging ability without requiring written answers.

Debugging exercises: Provide a working program and a broken version. Ask the student to identify the error using only their screen reader and debugging tools. This directly tests real-world skill.

Feature extension: Give a working program and ask the student to add a specific feature. This is a more realistic assessment than writing

from scratch and allows scaffolding.

Verbal design review: Present a programming problem and ask the student to describe their approach — data structures, functions, algorithm — without writing any code. Assess reasoning quality.

Portfolio review: Maintain a collection of all programs written during the course. Assess growth, consistency, commenting quality, and the student's ability to explain their own work.

Grading Considerations

- Do not penalize for missing visual formatting (e.g., a missing newline that only matters visually).
- Do reward for accessibility features the student adds above requirements: clear prompts, confirmation messages, help text.
- Recognize that a blind student may spend significantly more time debugging than a sighted peer on identical problems. Factor this into time expectations.
- Consider allowing oral explanations as alternatives to written code comments for students who find comment-writing slow.

---

18. Supporting Diverse Needs Within the Visually Impaired Population

For Students with Low Vision

- Use high-contrast VS Code themes: Dark High Contrast, Light High Contrast (available under Color Themes in settings).
- Increase editor font size: `Ctrl+Shift+P` -> "Preferences: Open Settings" -> `editor.fontSize`.
- Enable zoom magnification: `Ctrl++` to zoom the VS Code window.
- Test whether syntax highlighting is visible with their specific visual condition. Some students benefit from; others are confused by color information they cannot fully perceive.

For Students Who Are Deaf-Blind

- All communication with the teacher must be through the braille display or tactile signing — no spoken instruction.
- Screen reader audio verbosity settings must be managed by the student (since they cannot hear audio feedback during class).
- Consider pairing with a trained interpreter who knows both tactile communication and computing concepts.
- Review the Perkins School for the Blind's resources on deaf-blind education technology.

For Students with Physical Disabilities in Addition to Visual Impairment

- Switch access: VS Code is fully keyboard-navigable and can be operated with adapted keyboards, switch interfaces, or eye-gaze systems paired with virtual keyboards.
- Voice control: Dragon NaturallySpeaking and macOS Voice Control can both operate VS Code. Voice coding requires additional voice commands for symbols.
- Sip-and-puff devices: Can control keyboard input. Students using these will have different typing rates — extend all time-based tasks.

For Students with Cognitive Differences in Addition to Visual Impairment

- Reduce the amount of new vocabulary per session.
- Use consistent metaphors across the entire course — do not introduce a new metaphor for the same concept.
- Provide written summaries of every lesson that the student can review with their screen reader later.
- Break complex concepts into smaller steps; complete each step before introducing the next.
- Build in regular review sessions rather than only forward-moving instruction.

---

19. Teaching AI Tools Responsibly

Framing AI for the Classroom Establish a clear classroom policy before students begin using AI tools:

Permitted: Using AI to explain error messages, generate a first draft for the student to critique and improve, explain a concept a different way, suggest test cases to consider.

Not permitted: Submitting AI-generated code as completed work without reading and understanding every line, using AI to complete assessments that are meant to evaluate individual skill.

Required when AI is used: The student must be able to explain any AI-generated code they submit as if they wrote it themselves. Inability to explain code is evidence it was not understood.

AI Tools and Blind Students — Specific Considerations **Potential accessibility benefit:** AI tools like GitHub Copilot are text-based — suggestions appear as text that a screen reader can read. Copilot's Accessible View integration in VS Code is improving.

Potential accessibility risk: AI may generate code that is visually clear but auditorily confusing — complex nested structures, long lines, or dense comprehensions that are harder to navigate with a screen reader than to read with eyes.

Teaching point: Ask students to evaluate AI-generated code for screen-reader accessibility:

- Are variable names descriptive?
- Are long lines split into readable pieces?
- Are print statements and prompts clear and complete?
- Does the code have docstrings?

If not, the AI-generated code needs to be modified before use — regardless of whether it runs correctly.

Thought Experiments on AI

1. *"An AI writes code that solves your problem but you don't understand it. You submit it. It passes tests. Have you learned anything? What is the difference between 'having a solution' and 'knowing how to solve problems'?"*
2. *"AI code generators are trained on billions of lines of code written mostly by sighted programmers. What biases might this introduce in the code they generate? Think specifically about: variable names, output design, prompts, error messages, documentation."*
3. *"If AI can write most code, which skills become more valuable for future programmers? Consider: understanding requirements, debugging, testing, communicating about code, evaluating solutions, and accessibility design."*

---

20. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Python Crash Course, 3rd Ed. (Matthes)	Beginner	Print/ebook	Good; primary textbook for this curriculum
Python Programming: An Introduction to CS (Zelle)	Beginner	Print/ebook	Good; companion textbook
Automate the Boring Stuff with Python (Sweigart)	Beginner–Intermediate	Free HTML / print	Excellent; HTML version fully accessible
Think Python (Downey)	Beginner	Free HTML / print	Excellent HTML version
Learn Python 3 the Hard Way (Shaw)	Beginner	Print/ebook	Plain text exercises, highly accessible

Resource	Level	Format	Accessibility
Python for Everybody (Severance)	Beginner	Free HTML + video	Video lectures have transcripts; good accessibility
Fluent Python (Ramalho)	Advanced	Print/ebook	Dense but thorough; for motivated advanced students
Python Docs (python.org/3)	Reference	Free HTML	Excellent screen-reader accessibility; definitive

Online Courses and Platforms

Platform	Course	Screen Reader Compatibility	Notes
edX / CS50P (Harvard)	Introduction to Programming with Python	High — video transcripts, accessible exercises	Free audit. Very high quality.
Coursera	Python for Everybody (Michigan)	Medium — some interactive elements inaccessible	Free audit available
Codecademy	Learn Python 3	Partial — editor accessible, some UI not	Requires testing with specific screen reader
SoloLearn	Python Core	High — mobile app works well with VoiceOver/TalkBack	Good for mobile-first learners
freeCodeCamp	Python curriculum	Medium — varies by module	Free; large community
Exercism	Python Track	High — terminal-based, mentor feedback	Excellent for structured practice after basics
CodingBat	Python problems	High — plain HTML, browser-based	Good for logic practice; no complex UI
replit.com	Any Python project	Medium — test with student's setup	No-install option; accessibility variable

Video Resources

Resource	Description	Accessibility
CS50P (Harvard)	Full course lectures, free on YouTube and edX	Closed captions, transcripts; very high production quality
Corey Schafer Python tutorials (YouTube)	In-depth concept tutorials	Closed captions available
Tech With Tim Python (YouTube)	Project-based tutorials	Closed captions available
Sentdex Python tutorials (YouTube)	Large series from beginner to advanced	Captions available
Python.org Getting Started	Short official tutorials	Plain text, no video

Tools Specifically for Accessibility in Programming Education

Tool	Purpose	Notes
Python Tutor (pythontutor.com)	Step-by-step code execution with state display	Text-based state panel is screen-reader accessible
Mu Editor	Beginner-focused Python IDE	Simpler than VS Code; reasonable screen reader support
Thonny	Beginner IDE with step debugger	Variable viewer is navigable via keyboard
NVDA + Remote	Remote screen sharing for teacher support	Student's screen reader on their machine; teacher can see output
GitHub Codespaces	Cloud VS Code environment	Same accessibility as local VS Code; useful when local setup fails
Braille Institute Resources	General adaptive technology guidance	brailleinstitute.org
APH (American Printing House)	Educational materials for blind students	aph.org

Communities and Professional Organizations

Organization	Relevance
National Federation of the Blind (NFB) — Science & Engineering Division	Blind scientists and engineers; mentorship opportunities
American Council of the Blind (ACB) — Technology Interest Group	Accessible technology advocacy and resources
CSUN Assistive Technology Conference	Annual conference; many papers on computing education for blind students
SIGACCESS (ACM SIG on Computers and Accessibility)	Academic research on accessible computing
Ability Central	Technology resources for people with disabilities
Hadley Institute for the Blind	Professional development for educators of blind students
Perkins School for the Blind	Resources including STEM education for blind students
Carroll Center for the Blind	Technology training for blind adults; may have curriculum resources
Stack Overflow Blind/VI Community	Informal community; blind developers helping each other

---

21. Quick Reference Sheets

Error Message Reading Protocol When an error occurs, guide the student through this sequence:

```
1. Navigate to terminal output (Ctrl+` or click panel)
2. Read from the BOTTOM of the error output upward
3. Note the ERROR TYPE (last line): SyntaxError, NameError, TypeError, etc.
4. Note the ERROR MESSAGE: specific description of what went wrong
5. Note the LINE NUMBER in the traceback
6. Jump to that line with Ctrl+G -> [line number] -> Enter
7. Read the line and the lines immediately before and after it
8. Categorize the error (see table below)
9. Fix and re-run
```

Error Type Quick Reference

Error Type	Common Cause	First Thing to Check
`SyntaxError`	Missing `:`, `)`, `"`, `]`, or `=` instead of `==`	The exact column number in the error message
`IndentationError`	Inconsistent spaces/tabs	Line indentation; use `cat -A` or `show whitespace` in VS Code
`NameError`	Variable not defined, or misspelled	Check spelling; check if variable was defined before use
`TypeError`	Wrong data type	Use `type()` on each variable involved in the operation
`ValueError`	Right type, wrong value	Check what you're converting: `int('hello')` fails; `int('5')` works
`IndexError`	List index out of range	Print `len(list)` and check what index you're trying to use
`KeyError`	Dictionary key doesn't exist	Use `.get()` or check with `if key in dict` first
`FileNotFoundError`	File doesn't exist at that path	Print `os.getcwd()` and verify the file path
`AttributeError`	Called a method that doesn't exist	Check spelling and whether the object is the type you think it is
`ZeroDivisionError`	Dividing by zero	Add a check before the division: `if denominator != 0`
`RecursionError`	Infinite recursion	Verify base case exists and is reachable

```
# Instead of: print(my_list)
# Use:
print("Items in collection:")
for i, item in enumerate(my_list, 1):
    print(f"  {i}. {item}")
print(f"Total: {len(my_list)} items")

# Instead of: print(my_dict)
# Use:
print("Current settings:")
for key, value in my_dict.items():
    print(f"  {key}: {value}")

# Menu display:
print("\n--- Main Menu ---")
print("1. Add item")
print("2. View all items")
print("3. Search")
```

```
print("4. Delete item")
print("5. Save and quit")
print("------------------")
choice = input("Enter option number (1-5): ")

# Confirmation messages:
print(f"Contact '{name}' added successfully.")
print(f"Found {len(results)} matching records.")
print("Data saved to contacts.json.")
print("No matching records found.")
print("Error: Name field cannot be empty. Please try again.")
```

Accessible Output Patterns

VS Code Keyboard Shortcut Reference Card

```
NAVIGATION
  Ctrl+G          Go to specific line number
  Ctrl+Home       Jump to start of file
  Ctrl+End        Jump to end of file
  Ctrl+Shift+O    Go to function/class (symbol picker)
  F12             Go to Definition
  Alt+F12         Peek Definition
  Shift+F12       Find all References
  Ctrl+P          Quick open file by name

ERRORS AND DIAGNOSTICS
  F8              Next error/warning
  Shift+F8        Previous error/warning
  Ctrl+Shift+M    Open Problems panel
  Shift+Alt+F2    Accessible View (for hover/tooltip content)

EDITING
  Shift+Alt+F     Format document (auto-indent)
  Ctrl+/          Toggle comment on selected lines
  Tab             Indent selected lines
  Shift+Tab       Outdent selected lines
  Ctrl+D          Select next occurrence of word
  Ctrl+H          Find and replace

TERMINAL
  Ctrl+`          Toggle integrated terminal
  Ctrl+Shift+`    New terminal
  Up/Down arrows  Navigate command history
  Tab             Auto-complete filenames
  Ctrl+C          Stop running program
```

```
SCREEN READER
  Ctrl+Shift+P    Command Palette
  Ctrl+Shift+P -> "Toggle Screen Reader Optimized"
  Shift+Alt+F1    Accessibility help
```

Unit 0: Setting Up Your Accessible Python Development Environment

Overview

A detailed environment setup is not merely a technical prerequisite; it empowers students and reduces dependency on external assistance. For visually impaired students, relying on external help for every setup or environment issue can be disempowering and slow down the learning process. This unit provides extremely detailed, screen-reader-friendly instructions for environment setup.

Target Audience: Students with no prior programming experience, relying on screen readers for all computer interactions.

Lesson 0.1: Installing Python and Visual Studio Code

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Appendix A, pages 463–468

Installing Python 3 Python is not always installed on a computer by default. Download the latest version from `https://www.python.org/downloads/`.

Windows

1. Locate the downloaded `.exe` file (usually in your Downloads folder). Activate it.
2. **CRUCIAL STEP:** When the installer window appears, check the **"Add Python to PATH"** checkbox. Use Tab to navigate to this checkbox and press Spacebar to check it.
3. Navigate to "Install Now" and activate it.
4. Follow any further prompts. Activate "Close" when finished.

macOS

1. Locate the downloaded `.pkg` file. Activate it.
2. Activate "Continue" through the steps.
3. Agree to the license terms. Choose the installation location (default is fine).
4. Enter your administrator password when prompted.
5. Activate "Install" and then "Close" when finished.

Linux Python 3 is often pre-installed. Verify in a terminal:

```
python3 --version
```

If not installed (Debian/Ubuntu):

```
sudo apt update
sudo apt install python3
sudo apt install python3-pip
```

Verifying Python Installation Open a terminal and type:

```
python3 --version
```

You should hear the version number (e.g., "Python 3.10.12"). Also check pip:

```
pip3 --version
```

Installing Visual Studio Code

1. Navigate to https://code.visualstudio.com/download.
2. Download the installer for your OS.

Windows

- **CRUCIAL STEP:** On the "Select Additional Tasks" screen, check **"Add to PATH"**.
- Activate "Next" and then "Install".

macOS

- Drag the "Visual Studio Code" application to your "Applications" folder.

```
cd Downloads
sudo dpkg -i code_*.deb
```

Linux (Debian/Ubuntu)

```
code --version
```

Verify VS Code

Skills Checklist

☐ I can successfully download the Python 3 installer for my operating system.
☐ I can run the Python installer and correctly select the "Add Python to PATH" option.
☐ I can successfully download the VS Code installer for my operating system.
☐ I can run the VS Code installer and correctly select the "Add to PATH" option.
☐ I can open my computer's system terminal.
☐ I can verify Python's installation by running `python3 --version` in the terminal.
☐ I can verify VS Code's installation by running `code --version` in the terminal.

Lesson 0.2: Configuring Your Screen Reader for VS Code

Reading References

- Matthes: Appendix B, pages 469–476
- Matthes: Appendix C, pages 477–482

General Screen Reader Tips for VS Code **1. Use Screen Reader Mode**

- Press `Ctrl + Shift + P` to open the Command Palette.
- Type "Toggle Screen Reader Optimized" and press Enter.

2. Accessible View

- Press `Shift + Alt + F2` (Windows/Linux) or `Shift + Option + F2` (macOS).
- Opens a flat, text-based representation of complex UI elements.
- Press `Esc` to close.

3. Keyboard-First Navigation

- `Tab` and `Shift+Tab`: Move focus between UI elements.
- Arrow Keys: Navigate within active panes.

4. Status Bar The status bar at the bottom provides current line/column number, file type, active Python environment, and error/warning counts.

Screen Reader Indentation Configuration Python relies heavily on indentation. Configure your screen reader to announce indentation changes.

NVDA (Windows)

1. Press `NVDA Key + N` -> Preferences -> Settings.
2. Find "Document Formatting".
3. Locate "Indentation" and set it to **"Say all"** or **"Beeps"**.
4. Activate "OK".

JAWS (Windows)

1. Press `JAWS Key + J` -> Alt+O for Options -> B for Basics.
2. Find "Text Processing" settings.
3. Configure indentation announcement.

VoiceOver (macOS) VoiceOver generally handles indentation well by default, announcing "4 spaces" or "tab" as you navigate.

Skills Checklist

☐ I can identify and toggle VS Code's "Screen Reader Optimized Mode".
☐ I can open and close the "Accessible View".
☐ I can navigate between VS Code panes using Tab and Shift+Tab.
☐ I can navigate line by line and character by character within the editor.
☐ I can locate and listen to information in the VS Code Status Bar.
☐ I can configure my screen reader to announce indentation changes.

Lesson 0.3: Setting Up Your VS Code Workspace

Reading References

- Matthes: Appendix B, pages 469–476
- Matthes: Chapter 8, pages 149–152

Installing VS Code Extensions

1. Open the Extensions View: `Ctrl + Shift + X` (Windows/Linux) or `Command + Shift + X` (macOS).
2. Search and install **"Python"** by Microsoft — the official and most important Python extension.
3. Search and install **"Pylance"** — provides excellent IntelliSense and type checking.
4. Optional: **"Code Runner"** — allows running code files with `Ctrl+Alt+N`.

Creating Your First Workspace Folder **Windows:** Open File Explorer (`Windows Key + E`). Navigate to Documents. Press `Ctrl + Shift + N` to create a new folder named `PythonProjects`.

macOS: Open Finder. Navigate to Documents. Press `Command + Shift + N` to create `PythonProjects`.

Opening the Folder in VS Code **Method 1 (Recommended):** Open terminal, navigate to the folder, then:

```
code .
```

Method 2: In VS Code, press `Ctrl + K, Ctrl + O` to "Open Folder".

VS Code Navigation Shortcuts

Area	Shortcut
Explorer	`Ctrl + Shift + E`
Search	`Ctrl + Shift + F`
Extensions	`Ctrl + Shift + X`
Toggle Sidebar	`Ctrl + B`
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Previous Error	`Shift + F8`
Integrated Terminal	'Ctrl + "

Skills Checklist

☐ I can open the Extensions View in VS Code.
☐ I can search for and install the "Python" extension.
☐ I can search for and install the "Pylance" extension.
☐ I can create a new folder for Python projects.
☐ I can open a specific folder as a workspace in VS Code.
☐ I can navigate the Activity Bar, Explorer, Editor, and Panel using keyboard shortcuts.
☐ I can effectively use the Command Palette (`Ctrl + Shift + P`).

Lesson 0.4: Python Environments and Terminal Usage

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Chapter 8, pages 149–152
- Matthes: Chapter 11, pages 210–211

Creating Your First Python File

1. Ensure Explorer is focused: `Ctrl + Shift + E`.
2. Create new file: `Ctrl + N`.
3. Save: `Ctrl + S`. Name it `hello.py`.
4. Type:

```
print("Hello from VS Code!")
```

Running Python Scripts **From System Terminal:**

```
cd ~/Documents/PythonProjects
python3 hello.py
```

From VS Code Integrated Terminal:

1. Open terminal: 'Ctrl + "
2. The terminal opens in your project folder automatically.
3. Run: `python3 hello.py`

Python Virtual Environments Virtual environments keep project dependencies separate, preventing conflicts.

Create a virtual environment:

```
python3 -m venv .venv
```

Activate:

- Windows (Command Prompt): `.venv\Scripts\activate.bat`
- Windows (PowerShell): `.venv\Scripts\Activate.ps1`
- macOS/Linux: `source ./.venv/bin/activate`

You will hear `(.venv)` at the beginning of your prompt when active.

Deactivate:

```
deactivate
```

Select Interpreter in VS Code:

1. Press `Ctrl + Shift + P`.
2. Type "Python: Select Interpreter" and press Enter.
3. Select the one that says "Python 3.x.x (.venv)".

Install packages:

```
pip install requests
pip list
```

Skills Checklist

☐ I can create a new Python file (.py) in VS Code.
☐ I can write a simple `print()` statement.
☐ I can run a Python script from the system terminal.
☐ I can run a Python script from the integrated terminal in VS Code.
☐ I understand why virtual environments are useful.

☐ I can create, activate, and deactivate a virtual environment.
☐ I can select the correct Python interpreter in VS Code.
☐ I can install Python packages using pip.

Lesson 0.5: General Accessibility Tools and Strategies

VS Code Accessibility Features

- **Accessible View** (`Shift+Alt+F2`): Reveals hover information and error details.
- **Focus on Errors** (`F8` / `Shift+F8`): Jump between errors without visual scanning.
- **Go to Definition** (`F12`): Navigate to where a function or variable is defined.
- **Go to Reference** (`Shift+F12`): Find all usages of a symbol.
- **Go to Line/Column** (`Ctrl+G`): Jump to a specific line number.

Python Documentation The official Python documentation is well-structured and accessible: https://docs.python.org/3/

Unit 0 Completion Checklist

☐ Python 3 is installed and `python3 --version` works in your system terminal.
☐ VS Code is installed and `code --version` works in your system terminal.
☐ Your screen reader is configured to announce indentation changes in VS Code.
☐ You can open your PythonProjects folder in VS Code.
☐ You have installed the official "Python" VS Code extension.
☐ You can create a new .py file, write `print("Hello!")`, and save it.
☐ You can run your hello.py file from both terminals.
☐ You can create and activate a virtual environment (.venv).
☐ You can select the virtual environment interpreter in VS Code.
☐ You can install a package using pip.
☐ You know how to use `Ctrl + Shift + P` (Command Palette) and `F8` (Go to next error).

Congratulations! You now have a powerful and accessible development environment ready for your Python journey.

Unit 1: Foundations of Computing and Getting Started with Python

Overview

This unit covers basic computer science concepts, setting up the Python environment, and writing and executing simple Python programs.

Objectives:

- Understand what computer science is and how hardware relates to software.
- Write and run your first Python program.
- Use variables to store and manipulate simple data types.

Lesson 1.1: Welcome to Computer Science

Reading References

- Zelle: Chapter 1, pages 1–7
- Matthes: Chapter 1, pages 3–14

What Is Computer Science? A computer is a **universal machine** — it can be programmed to solve any problem that can be broken down into a series of precise, logical steps. The true power of a computer doesn't come from its hardware but from the software we create.

Computer science is the study of problem-solving, not just programming. It's an interdisciplinary field involving breaking down complex problems, designing algorithms, and implementing efficient solutions.

Hardware vs. Software

Component	Description
Hardware	Physical components you can touch: keyboard, screen, processor, memory
Software	Instructions that direct hardware: programs, operating systems, apps
CPU	The processor that executes instructions
Memory (RAM)	Temporary storage for active programs and data
Storage	Permanent storage for files

Analogy: Think of a kitchen. The stove, refrigerator, and pots are **hardware**. A cookie recipe is **software** — it provides precise steps for the "hardware" (the cook) to follow.

Why Python? Python is an ideal language for beginners because of its:

- **Readability** — English-like syntax makes code easy to understand.
- **Simplicity** — Less "boilerplate" code than other languages.
- **Versatility** — Used in web development, data science, AI, and more.

The Problem-Solving Approach

1. Understand the problem.
2. Break it down into smaller pieces.
3. Design a step-by-step solution (an algorithm).
4. Implement the solution in Python.
5. Test and refine your code.

Activities **Activity 1.1.1: "Human Computer" Role-Play**

- In pairs, one student is the "computer" and the other is the "programmer."
- The programmer provides simple, precise instructions for a common task (e.g., "make a peanut butter and jelly sandwich").
- The "computer" follows only what is literally said.
- Purpose: Illustrate the need for precise, unambiguous instructions.

Activity 1.1.2: Identify Hardware and Software

- List five items you use daily.
- For each, identify whether it is hardware or software and explain your choice.

Skills Checklist

☐ I can define what computer science is.
☐ I can distinguish between computer hardware and software.
☐ I can explain what a computer program is.
☐ I can identify Python as a programming language.

Lesson 1.2: Your First Python Program — "Hello World!"

Reading References

- Zelle: Chapter 1, pages 7–13
- Matthes: Chapter 1, pages 3–14

The `print()` Function The `print()` function is Python's primary method for displaying output to the user.

```
print("Hello, World!")
```

Expected output:

```
Hello, World!
```

Key Concepts

- **Strings** are text data enclosed in quotation marks (single or double).
- **Comments** start with `#` and are ignored by Python.
- Python executes code **line by line**, from top to bottom.

Examples **Example 1.2.1: Simple Greeting**

```
# This program prints a classic greeting.
print("Hello, World!")
```

Example 1.2.2: Personalized Message

```
# This program prints a message with your name.
print("Hello, Alice!")
print("Welcome to Python programming!")
```

Expected output:

```
Hello, Alice!
Welcome to Python programming!
```

```
print("Hello, World"        # Missing closing parenthesis --- SyntaxError
print "Hello"               # Python 2 style --- SyntaxError in Python 3
Print("Hello")              # Capital P --- NameError
```

Common Mistakes

Activities **Activity 1.2.1: My First Message**

1. Open VS Code and confirm the PythonProjects folder is open.
2. Create a new file as `my_message.py`.
3. Type:

```
print("My favorite color is blue.")
print("My favorite animal is a dog.")
```

4. Save and run the file.

Activity 1.2.2: Exploring Print Statements

1. Experiment with:
   - Printing numbers: `print(123)`
   - Printing sentences with punctuation.
   - Omitting quotes: `print(Hello)` — expect a `NameError`.

Skills Checklist

☐ I can write a `print()` statement in Python.
☐ I can save a Python file with the `.py` extension.
☐ I can execute a Python script from the terminal.
☐ I understand that the output of `print()` appears in the terminal.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Zelle: Chapter 2, pages 24–26
- Matthes: Chapter 2, pages 15–32

What Is a Variable? A **variable** is a name that refers to a value stored in the computer's memory. The **assignment operator** `=` creates a connection between a name and a value.

```
age = 25
name = "Alice"
height = 5.6
```

Rules for variable names:

- Must start with a letter or underscore
- Can only contain letters, digits, and underscores
- Are case-sensitive (`age` is different from `Age`)
- Cannot be a Python keyword (`if`, `for`, `while`, etc.)

Best practices:

- Use descriptive names: `student_count`, not `sc`
- Use lowercase with underscores: `first_name`

Essential Data Types

Type	Description	Examples
`str` (String)	Text data	`"hello"`, `"Python"`

Type	Description	Examples
`int` (Integer)	Whole numbers	`10`, `-5`, `0`
`float` (Float)	Numbers with decimals	`3.14`, `9.99`
`bool` (Boolean)	True or False	`True`, `False`

Examples **Example 1.3.1: Storing and Printing a String**

```
student_name = "Charlie"
print("Hello, " + student_name + "!")
```

Output:

```
Hello, Charlie!
```

Example 1.3.2: Storing and Printing Numbers

```
apple_count = 5
orange_count = 3
total_fruit = apple_count + orange_count
print("You have", total_fruit, "fruits in total.")

price_per_item = 2.50
quantity = 4
total_cost = price_per_item * quantity
print("The total cost is:", total_cost)
```

Output:

```
You have 8 fruits in total.
The total cost is: 10.0
```

```
2nd_number = 10          # Invalid variable name --- starts with digit
print("Result: " + 42)   # TypeError --- can't concatenate str and int
```

Common Errors

Activities **Activity 1.3.1: Personal Information Variables**

```
my_name = "Jamie"
favorite_number = 7
height_in_meters = 1.75
favorite_food = "pizza"

print("My name is:", my_name)
print("My favorite number is:", favorite_number)
print("My height is:", height_in_meters, "meters")
print("My favorite food is:", favorite_food)
```

Activity 1.3.2: Simple Math Challenge

```
num1 = 15
num2 = 7
sum_result = num1 + num2
difference = num1 - num2
product = num1 * num2
quotient = num1 / num2

print("Sum:", sum_result)
print("Difference:", difference)
print("Product:", product)
print("Quotient:", quotient)
```

Key Table: Python Data Types and Operations

Data Type	Description	Example Value	Common Operations
`str`	Text, words, sentences	`"Hello"`, `"Python"`	`+` (concatenation), `len()`, `.upper()`
`int`	Whole numbers	`10`, `-5`, `0`	`+`, `-`, `*`, `/`, `%`
`float`	Numbers with decimals	`3.14`, `9.99`, `0.0`	`+`, `-`, `*`, `/`

Skills Checklist

- ☐ I can define what a variable is in programming.
- ☐ I can create a valid variable name in Python.
- ☐ I can assign string, integer, and float values to variables.
- ☐ I can use `print()` to display the value of a variable.
- ☐ I can perform basic arithmetic operations with numbers in Python.

Unit 2: Writing Simple Programs and Numeric Computations

Overview

This unit explores the software development process, more complex numeric operations, and how to obtain and use input from users.

Objectives:

- Understand the phases of software development.
- Work with different numeric data types (integers, floats).
- Perform type conversions.
- Use mathematical operators and the math library.
- Get user input with `input()`.

Lesson 2.1: The Software Development Process

Reading References

- Zelle: Chapter 2, pages 21–22 and pages 35–36

The Three-Phase Development Process **1. Design (Plan first)** Before writing any code, fully understand the problem:

- Identify **inputs** (information the program needs) and **outputs** (results it must produce).
- Write an **algorithm** — a step-by-step plan in plain English (**pseudocode**).
- Consider **edge cases** — unusual inputs that might break your program.

2. Implement (Write the code)

- Translate your design into Python syntax.
- Code **incrementally** — build in small pieces and test as you go.
- Follow coding standards and use descriptive variable names.

3. Test and Debug (Verify it works)

- Run with a variety of inputs, including normal values and edge cases.
- Read error messages carefully — they tell you the type of error and the line number.
- Use `print()` statements to check variable values during debugging.

Why This Process Matters

- **Reduces frustration** — Having a plan prevents getting stuck.
- **Improves code quality** — A thoughtful design leads to cleaner code.
- **Simplifies debugging** — Test each component in isolation.
- **Saves time** — Good design prevents time-consuming rewrites.

Activities **Activity 2.1.1: Planning a Tip Calculator** Write pseudocode before coding:

```
1. GET bill amount from user
2. CALCULATE tip as 15% of bill amount
3. CALCULATE total amount (bill + tip)
4. PRINT tip amount
5. PRINT total amount
```

Activity 2.1.2: Tracing a Simple Program Without running the code, trace the execution and predict output:

```
a = 10
b = 5
c = a * b
a = a + 2
print(c)
print(a)
```

Skills Checklist

☐ I can describe the purpose of the software development process.
☐ I can list and briefly explain the main phases of software development.
☐ I understand that problem-solving and design happen before writing code.

Lesson 2.2: Elements of Programs — Expressions and Assignment

Reading References

- Zelle: Chapter 2, pages 24–26

Expressions and Assignment An **expression** is any combination of values, variables, and operators that evaluates to a single result. Python follows the standard mathematical order of operations (PEMDAS).

```
result1 = 10 / 2 + 3  # 10/2 is 5, then 5+3 = 8.0
message = "Python" + " " + "is fun!"
```

Assignment:

```
score = 50
score = score + 20  # score is now 70
```

Examples **Example 2.2.1: Expression Evaluation**

```
result1 = 10 / 2 + 3
print("Result 1:", result1)   # Output: Result 1: 8.0

message = "Python" + " " + "is fun!"
print("Message:", message)    # Output: Message: Python is fun!
```

Example 2.2.2: Reassigning Variables

```
score = 50
print("Initial score:", score)
score = score + 20
print("New score:", score)     # Output: New score: 70
```

Skills Checklist

☐ I can identify an expression in Python code.
☐ I can predict the result of simple Python expressions.
☐ I can explain how the assignment operator = works.
☐ I can demonstrate how to update a variable's value.

Lesson 2.3: Input and Output Revisited

Reading References

- Zelle: Chapter 2, pages 27–33
- Matthes: Chapter 7, pages 114–116

The `input()` Function The `input()` function pauses program execution and waits for the user to type something and press Enter. **It always returns a string.**

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

F-Strings (Formatted String Literals) F-strings provide a clean way to embed variable values directly into a string:

```
name = "Alice"
age = 30
print(f"Hello, {name}! You are {age} years old.")
```

Converting Input for Calculations Since `input()` always returns a string, you must convert it for math:

```
age_str = input("How old are you? ")
age_int = int(age_str)
print(f"Next year you will be {age_int + 1} years old.")
```

Examples **Example 2.3.1: Basic Input**

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

Example 2.3.2: Numeric Input and Calculation

```
num1_str = input("Enter the first number: ")
num1 = int(num1_str)

num2_str = input("Enter the second number: ")
num2 = int(num2_str)

sum_result = num1 + num2
print(f"The sum of {num1} and {num2} is {sum_result}.")
```

Activities **Activity 2.3.1: Temperature Converter**

```
fahrenheit_str = input("Enter temperature in Fahrenheit: ")
fahrenheit = float(fahrenheit_str)
celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit}°F is {celsius:.2f}°C")
```

Skills Checklist

- ☐ I can use the `input()` function to ask the user for information.
- ☐ I can provide a clear prompt message when using `input()`.
- ☐ I remember that `input()` returns a string, even for numbers.
- ☐ I can combine text and variable values in a `print()` statement using f-strings.

Lesson 2.4: Computing with Numbers and Type Conversions

Reading References

- Zelle: Chapter 3, pages 41–48
- Matthes: Chapter 2, pages 26–28

Integers and Floats

Type	Description	Example
`int`	Whole numbers (exact)	`5`, `-20`, `1000000`
`float`	Numbers with decimals	`3.14`, `-2.5`, `6.0`

Key behaviors:

- Division `/` always returns a float: `10 / 2` -> `5.0`
- Floor division `//` returns the integer part: `10 // 3` -> `3`
- Mixed operations promote to float: `10 + 3.5` -> `13.5`

```
int("123")      # -> 123  (truncates: int(3.9) -> 3)
float("3.14")   # -> 3.14
str(42)         # -> "42"
```

Type Conversion Functions

Examples **Example 2.4.1: Type Conversion**

```
num_str = "123"
num_int = int(num_str)
print(f"String '{num_str}' as integer: {num_int}, type: {type(num_int)}")

num_float = float(num_str)
print(f"String '{num_str}' as float: {num_float}, type: {type(num_float)}")

my_number = 42
my_text = str(my_number)
print(f"Integer {my_number} as string: '{my_text}', type: {type(my_text)}")
```

Common Type Error:

```
# This will cause a TypeError:
# print("My age is " + 14)
# Fix: print("My age is " + str(14))
# Or: print(f"My age is {14}")
```

Activities **Activity 2.4.1: Mixed Bag Calculator**

```
age_str = input("Enter your age: ")
age = int(age_str)

weight_str = input("Enter your weight in kg: ")
weight = float(weight_str)

phrase = input("Enter a short phrase: ")
```

```
print(f"Age doubled: {age * 2}")
print(f"Weight + age: {weight + age}")
print(f"Phrase length: {len(phrase)} characters")
```

Skills Checklist

☐ I can identify when to use an integer versus a float.
☐ I can convert a string to an integer using `int()`.
☐ I can convert a string to a float using `float()`.
☐ I can convert a number to a string using `str()`.
☐ I can apply type conversions to perform calculations on user input.
☐ I understand and can apply the order of operations.

Lesson 2.5: Using the Math Library

Reading References

- Zelle: Chapter 3, pages 49–51

```
import math
```

Importing the Math Module

Key Math Functions and Constants

Function/Constant	Description	Example
`math.pi`	π ≈ 3.14159	`math.pi`
`math.e`	e ≈ 2.71828	`math.e`
`math.sqrt(x)`	Square root of x	`math.sqrt(25)` -> `5.0`
`math.pow(x, y)`	x raised to power y	`math.pow(2, 4)` -> `16.0`
`math.ceil(x)`	Round up	`math.ceil(4.2)` -> `5`
`math.floor(x)`	Round down	`math.floor(4.8)` -> `4`
`math.log(x, base)`	Logarithm	`math.log(100, 10)` -> `2.0`
`math.sin(x)`	Sine (in radians)	`math.sin(math.pi/2)` -> `1.0`
`math.radians(deg)`	Convert degrees to radians	`math.radians(90)`

Examples **Example 2.5.1: Circle Calculations**

```
import math

radius = 5
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area of circle with radius {radius}: {area:.2f}")
print(f"Circumference: {circumference:.2f}")

number = 81
square_root = math.sqrt(number)
print(f"Square root of {number}: {square_root}")
```

Activities **Activity 2.5.1: Circle Calculator**

```
import math

radius_str = input("Enter the radius: ")
radius = float(radius_str)

area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
```

Activity 2.5.2: Distance Finder

```
import math

x = float(input("Enter x coordinate: "))
y = float(input("Enter y coordinate: "))
distance = math.sqrt(x**2 + y**2)
print(f"Distance from origin: {distance:.2f}")
```

Skills Checklist

☐ I can explain what a Python module is.
☐ I can import the `math` module into my program.
☐ I can use `math.pi` and `math.sqrt()`.
☐ I can use `math.ceil()` to round up and `math.floor()` to round down.

Unit 3: Control Flow — Decisions and Loops

Overview

This unit introduces how programs make decisions using `if` statements and perform repetitive tasks with `for` loops, along with fundamental string manipulation techniques.

Objectives:

- Control program flow with `if`, `elif`, and `else`.
- Iterate over sequences using `for` loops.
- Understand and manipulate strings.

Lesson 3.1: Boolean Expressions and `if` Statements

Reading References

- Zelle: Chapter 7, pages 165–173
- Matthes: Chapter 5, pages 71–78

Comparison and Boolean Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 10`	`True`
`<`	Less than	`5 < 10`	`True`
`>`	Greater than	`10 > 5`	`True`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal	`10 >= 5`	`True`

Boolean Operator	Description	Example	Result
`and`	True if both are true	`True and False`	`False`
`or`	True if at least one is true	`True or False`	`True`
`not`	Inverts the truth value	`not True`	`False`

```
age = 15
if age >= 18:
    print("You are old enough to vote.")
print("Program finished.")  # This runs regardless
```

The `if` Statement **Indentation defines which lines belong to the `if` block.**

Examples **Example 3.1.1: Simple Age Check**

```
age = 17
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Example 3.1.2: Password Length Check

```
password = "mysecret"
if len(password) < 8:
    print("Weak password: Must be at least 8 characters.")
```

Activities **Activity 3.1.1: Number Comparison Game**

```
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

if num1 > num2:
    print(f"{num1} is greater than {num2}.")
if num1 == num2:
    print("The numbers are equal.")
if num1 < num2:
    print(f"{num1} is less than {num2}.")
```

Skills Checklist

☐ I can identify `True` and `False` as Boolean values.
☐ I can use comparison operators to create Boolean expressions.
☐ I can write a basic `if` statement.
☐ I can correctly indent the code block belonging to an `if` statement.

Lesson 3.2: Chained Conditionals (`elif`, `else`)

Reading References

- Zelle: Chapter 7, pages 173–176
- Matthes: Chapter 5, pages 79–84

`if-elif-else` Structure Python checks each condition in **sequential order**. As soon as a condition is `True`, it runs that block and skips the rest.

```
score = 85

if score >= 90:
```

```
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Output: `Grade: B`

Examples **Example 3.2.1: Stages of Life**

```
age = 25
if age < 2:
    print("You are a baby.")
elif age < 4:
    print("You are a toddler.")
elif age < 13:
    print("You are a kid.")
elif age < 20:
    print("You are a teenager.")
else:
    print("You are an adult.")
```

Activities **Activity 3.2.1: Movie Ticket Pricing**

```
age = int(input("Enter your age: "))

if age <= 5:
    print("Ticket price: Free")
elif age <= 12:
    print("Ticket price: $7")
elif age <= 64:
    print("Ticket price: $12")
else:
    print("Ticket price: $5")
```

Skills Checklist

☐ I can use an `else` statement to provide an alternative path.
☐ I can use `elif` to check multiple conditions in order.
☐ I can use `and`, `or`, and `not` operators to combine conditions.

Lesson 3.3: Introduction to Lists

Reading References

- Zelle: Chapter 5, pages 105–106
- Matthes: Chapter 3, pages 33–47

```
colors = ["red", "blue", "green", "yellow"]

print(colors[0])   # "red"   (first item)
print(colors[-1])  # "yellow" (last item)
print(len(colors)) # 4
```

Creating and Accessing Lists **Zero-based indexing:** The first item is at index `0`.

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "grape"  # Change "banana" to "grape"
print(fruits)         # ["apple", "grape", "cherry"]
```

Modifying Lists

Activities **Activity 3.3.1: My Top 3 Hobbies**

```
hobbies = ["reading", "coding", "hiking"]
print("My hobbies:", hobbies)
print("My second hobby:", hobbies[1])
hobbies[2] = "swimming"
print("Updated hobbies:", hobbies)
```

Skills Checklist

☐ I can create a list in Python.
☐ I can access individual items in a list using their index.
☐ I can access list items from the end using negative indices.
☐ I can change the value of an item at a specific index.

Lesson 3.4: Iterating with `for` Loops

Reading References

- Zelle: Chapter 2, pages 34–36
- Matthes: Chapter 4, pages 49–60

```
for item in sequence:
    # code that runs for each item
```

The `for` Loop Structure

Examples **Example 3.4.1: Looping Through a List**

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}.")
print("Finished listing fruits.")
```

Example 3.4.2: Looping Through a String

```
word = "Python"
for char in word:
    print(char)
```

Example 3.4.3: Using `range()`

```
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):   # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
    print(i)
```

Activities **Activity 3.4.1: My Favorite Things**

```
favorite_things = ["sunshine", "coffee", "books", "music", "mountains"]
for thing in favorite_things:
    print(f"I love {thing}!")
```

Activity 3.4.2: Countdown Timer

```
start = int(input("Enter a starting number: "))
for i in range(start, 0, -1):
    print(i)
print("Blast off!")
```

Skills Checklist

☐ I can explain the purpose of a `for` loop.
☐ I can write a `for` loop to iterate over items in a list.
☐ I can write a `for` loop to iterate over characters in a string.
☐ I can use the `range()` function to control how many times a loop runs.

Lesson 3.5: Basic String Operations

Reading References

- Zelle: Chapter 5, pages 99–115
- Matthes: Chapter 2, pages 19–25

```
message = "  Hello Python!  "

message.strip()           # "Hello Python!"   --- removes whitespace
message.upper()           # "  HELLO PYTHON!  "
message.lower()           # "  hello python!  "
message.replace("Hello", "Hi")  # "  Hi Python!  "
len(message)              # 17
```

Common String Methods **Important:** String methods return a **new string** — they do not modify the original.

Examples **Example 3.5.1: String Manipulation**

```
message = "  Hello Python  "
print(f"Original: '{message}'")
print(f"Length: {len(message)}")
print(f"Uppercase: '{message.upper()}'")
print(f"Stripped: '{message.strip()}'")
```

Example 3.5.2: Combining Strings

```
first_name = "Alice"
last_name = "Smith"
full_name = first_name + " " + last_name
print(f"Full name: {full_name}")
```

Activities **Activity 3.5.1: Name Formatter**

```
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
full_name = first_name + " " + last_name
print(f"Full name: {full_name}")
print(f"Uppercase: {full_name.upper()}")
print(f"Total characters: {len(full_name)}")
```

Skills Checklist

☐ I can combine strings using concatenation.
☐ I can find the length of a string using `len()`.
☐ I can convert a string to uppercase/lowercase.
☐ I can remove extra spaces using `.strip()`.
☐ I can replace parts of a string using `.replace()`.

Unit 4: Data Structures — Lists and Dictionaries

Overview

This unit deepens understanding of lists and introduces dictionaries as powerful mechanisms for organizing data.

Objectives:

- Add, remove, and organize list elements.
- Work with tuples (immutable lists).
- Use dictionaries for key-value pair storage.
- Loop through dictionaries.

Lesson 4.1: More on Lists

```
my_list = [10, 20, 30]

my_list.append(40)          # Add to end -> [10, 20, 30, 40]
my_list.insert(1, 15)       # Insert at index 1 -> [10, 15, 20, 30, 40]
del my_list[0]              # Delete by index -> [15, 20, 30, 40]
removed = my_list.pop()     # Remove and return last -> removed=40, list=[15, 20, 30]
my_list.remove(20)          # Remove by value -> [15, 30]
```

Adding and Removing Elements

```
todo_list = ["Buy groceries", "Walk the dog"]

todo_list.append("Call grandma")
todo_list.insert(0, "Wake up early")
del todo_list[1]
completed_task = todo_list.pop()
todo_list.remove("Walk the dog")
print(todo_list)
```

Example

Skills Checklist

☐ I can add an item to the end of a list using `.append()`.
☐ I can insert an item at a specific position using `.insert()`.
☐ I can remove an item by index using `del`.
☐ I can remove an item and retrieve its value using `.pop()`.
☐ I can remove the first occurrence of a value using `.remove()`.

Lesson 4.2: Organizing Lists

```
numbers = [5, 2, 9, 1]
numbers.sort()                          # Permanent sort in-place
print(numbers)                          # [1, 2, 5, 9]

fruits = ["orange", "apple", "banana"]
sorted_fruits = sorted(fruits)          # Temporary sort --- original unchanged
print(fruits)                           # ['orange', 'apple', 'banana']
print(sorted_fruits)                    # ['apple', 'banana', 'orange']

numbers.reverse()                       # Reverse in-place
numbers.sort(reverse=True)              # Sort descending
print(len(numbers))                     # Length
```

Sorting and Reversing

Skills Checklist

☐ I can sort a list permanently using `.sort()`.
☐ I can get a temporarily sorted version using `sorted()`.
☐ I can reverse a list using `.reverse()`.
☐ I can find how many items are in a list using `len()`.

Lesson 4.3: Tuples — Immutable Lists

Tuples Tuples are defined with parentheses and **cannot be changed** after creation.

```
coordinates = (10, 20, 30)
print(coordinates[0])     # 10
print(coordinates[-1])    # 30

# This would cause a TypeError:
# coordinates[1] = 25
```

When to use tuples: For data that should remain constant (coordinates, RGB values, fixed settings).

Skills Checklist

☐ I can create a tuple in Python.
☐ I can explain the main difference between a list and a tuple (mutability).
☐ I can access items in a tuple by their index.

Lesson 4.4: Introducing Dictionaries

Reading References

- Zelle: Chapter 11, pages 307–309
- Matthes: Chapter 6, pages 91–98

Dictionary Basics A dictionary stores **key-value pairs**. Keys must be unique and immutable (strings, numbers, tuples). Values can be anything.

```
student_profile = {
    "name": "Emily",
    "age": 13,
    "grade": "7th",
    "favorite_subject": "Science"
}

print(student_profile["name"])      # "Emily"
student_profile["city"] = "Springfield"   # Add new key
student_profile["age"] = 14               # Modify existing key
del student_profile["grade"]              # Remove key
```

Examples **Example 4.4.1: Book Information Dictionary**

```
my_book = {
    "title": "The Hitchhiker's Guide to the Galaxy",
    "author": "Douglas Adams",
    "publication_year": 1979,
    "genre": "Science Fiction"
}

print(f"Title: {my_book['title']}")
print(f"Author: {my_book['author']}")

my_book["number_of_pages"] = 193
my_book["genre"] = "Comedy Science Fiction"
print(my_book)
```

Skills Checklist

☐ I can create a dictionary in Python.
☐ I can explain what a key-value pair is.
☐ I can access a value in a dictionary using its key.
☐ I can add, modify, and remove key-value pairs.

Lesson 4.5: Looping Through Dictionaries

```
inventory = {"apples": 50, "bananas": 30, "oranges": 40}

# Loop through keys (default)
for item_name in inventory:
    print(item_name)

# Loop through values only
for count in inventory.values():
    print(count)

# Loop through key-value pairs
for item_name, count in inventory.items():
    print(f"{item_name}: {count} in stock")
```

Dictionary Iteration Methods

Skills Checklist

☐ I can loop through all the keys in a dictionary.
☐ I can loop through all the values in a dictionary.
☐ I can loop through both keys and values at the same time.

Unit 5: User Input and `while` Loops

Overview

This unit focuses on obtaining diverse input from users and controlling program execution through indefinite loops.

Objectives:

- Implement indefinite loops using `while` statements.
- Control `while` loop execution with `break` and `continue`.
- Avoid infinite loops.

Lesson 5.1: How the `input()` Function Works

Key Reminder `input()` **always returns a string**. Convert to numeric types as needed.

```
age_input = input("How old are you? ")
print(f"Type: {type(age_input)}")    # <class 'str'>

age_int = int(age_input)
print(f"In 5 years: {age_int + 5}")
```

Skills Checklist

☐ I can effectively use `input()` to get user data.
☐ I can confirm that `input()`'s return value is always a string.
☐ I can convert user input to a numeric type when necessary.

Lesson 5.2: The `while` Loop in Action

`for` vs. `while` Loop Comparison

Feature	`for` Loop	`while` Loop
Purpose	Definite iteration (known count)	Indefinite iteration (condition-based)
When to Use	Iterating over lists, ranges	Repeating until a condition is met
Control Variable	Automatically updated	Must be manually updated
Risk	Less prone to infinite loops	High risk if condition never becomes false

Examples **Example 5.2.1: Simple Counter**

```
count = 1
while count <= 5:
    print(count)
    count += 1
print("Counting finished!")
```

Example 5.2.2: User Input Loop (Sentinel Value)

```
message = ""
while message != 'quit':
    message = input("Enter a message, or type 'quit' to exit: ")
    if message != 'quit':
        print(f"You said: {message}")
print("Goodbye!")
```

Skills Checklist

☐ I can explain the difference between a `for` loop and a `while` loop.
☐ I can write a basic `while` loop.
☐ I can make sure my `while` loop has a way to stop.

Lesson 5.3: Using Flags and `break` to Exit Loops

```
active = True
while active:
    message = input("Say something (or 'quit' to stop): ")
    if message == 'quit':
        active = False
    else:
        print(f"You said: {message}")
```

Flag Variable

```
while True:
    city = input("Enter a city (or 'done' to finish): ")
    if city == 'done':
        break
    print(f"I've heard {city} is a great place!")
```

`break` Statement

Skills Checklist

☐ I can use a Boolean "flag" variable to control a `while` loop.
☐ I can use the `break` statement to exit a loop early.

Lesson 5.4: Using `continue` in Loops

```
number = 0
while number < 10:
    number += 1
    if number % 2 == 0:
        continue        # Skip even numbers
    print(number)       # Prints 1, 3, 5, 7, 9
```

`continue` — Skip the Current Iteration

`continue` vs. `break`

- `break` — **exits the loop entirely**
- `continue` — **skips only the current iteration**, loop continues

Skills Checklist

☐ I can use `continue` to skip the rest of the current loop iteration.
☐ I can identify scenarios where `continue` would be useful.

Lesson 5.5: Avoiding Infinite Loops

An **infinite loop** never ends because its condition always remains `True`.

Common causes:

1. Forgetting to update the loop variable.
2. Updating the variable in the wrong direction.
3. Using an incorrect logical condition.

```
# INFINITE LOOP --- Don't run!
count = 1
while count <= 5:
    print(count)
    # Missing: count += 1

# CORRECT
count = 1
while count <= 5:
    print(count)
    count += 1
```

To stop a running infinite loop: Press `Ctrl + C` in the terminal.

Skills Checklist

☐ I can define what an infinite loop is.
☐ I can identify common causes of infinite loops.
☐ I know how to stop a running program in an infinite loop.

Unit 6: Functions

Overview

This unit introduces functions, enabling organization of code into reusable blocks, passing arguments, and returning values.

Objectives:

- Define and call functions.
- Pass arguments to functions.
- Return values from functions.
- Understand the concept of scope.

Lesson 6.1: Defining Functions

Reading References

- Zelle: Chapter 6, pages 137–142
- Matthes: Chapter 8, pages 129–130

```
def function_name(parameter1, parameter2):
    """Docstring describing what the function does."""
    # Function body
    print(f"Hello, {parameter1}!")
```

Function Syntax

Examples **Example 6.1.1: Simple Greeting Function**

```
def greet_user():
    """Prints a simple greeting message."""
    print("Hello, Python programmer!")

greet_user()   # Call the function
greet_user()   # Call it again
```

Example 6.1.2: Function with a Parameter

```
def greet_name(name):
    """Prints a personalized greeting."""
    print(f"Hello, {name}!")

greet_name("Alice")
greet_name("Bob")
```

Skills Checklist

☐ I can explain why functions are useful.
☐ I can define a simple function in Python.
☐ I can use parameters to specify function inputs.
☐ I can call a function by its name.

Lesson 6.2: Passing Arguments

```
def describe_animal(animal_type, animal_name):
    print(f"I have a {animal_type} named {animal_name}.")
```

```
# Positional arguments (order matters)
describe_animal("dog", "Buddy")

# Keyword arguments (order doesn't matter)
describe_animal(animal_name="Max", animal_type="cat")
```

Positional vs. Keyword Arguments

```
def make_shirt(size, message="I love Python"):
    print(f"Making a {size}-sized shirt: '{message}'")

make_shirt("large")                      # Uses default message
make_shirt("medium", "Hello World")      # Custom message
```

Default Values

Skills Checklist

☐ I can differentiate between a parameter and an argument.
☐ I can pass arguments using positional order.
☐ I can pass arguments using keyword arguments.

Lesson 6.3: Return Values

```
def add_numbers(num1, num2):
    """Adds two numbers and returns the result."""
    total = num1 + num2
    return total

result = add_numbers(5, 3)
print(f"The sum is: {result}")    # 8
```

The `return` Statement **Key difference:** `print()` displays a value; `return` sends a value back to the caller for further use.

Skills Checklist

☐ I can use `return` to send a value back from a function.
☐ I can explain the difference between `print()` and `return`.
☐ I can store the returned value of a function in a variable.

Lesson 6.4: Functions with Lists and Dictionaries

Since lists are **mutable**, modifying them inside a function changes the original:

```
def add_new_user(user_list, new_user):
    user_list.append(new_user)

users = ['Alice', 'Bob']
add_new_user(users, 'Charlie')
print(users)   # ['Alice', 'Bob', 'Charlie']
```

To prevent modification, pass a copy:

```
add_new_user(users[:], 'Charlie')   # Original unchanged
```

Skills Checklist

☐ I can pass a list or dictionary as an argument to a function.
☐ I understand that modifying a list inside a function affects the original.
☐ I can create a copy of a list to avoid unintended modifications.

Lesson 6.5: Arbitrary Number of Arguments

```
def make_pizza(size, *toppings):
    print(f"Making a {size}-inch pizza with:")
    for topping in toppings:
        print(f"  - {topping}")

make_pizza(12, 'pepperoni')
make_pizza(16, 'mushrooms', 'green peppers', 'extra cheese')
```

`*args` — Arbitrary Positional Arguments

```
def build_profile(first, last, **user_info):
    user_info['first_name'] = first
    user_info['last_name'] = last
    return user_info

profile = build_profile('albert', 'einstein',
                        location='princeton',
                        field='physics')
print(profile)
```

`**kwargs` — Arbitrary Keyword Arguments

Skills Checklist

- ☐ I can define a function using `*args` for arbitrary positional arguments.
- ☐ I can define a function using `**kwargs` for arbitrary keyword arguments.

Unit 7: Classes and Object-Oriented Programming

Overview

This unit introduces the fundamental concepts of Object-Oriented Programming (OOP) in Python, focusing on defining and using classes, attributes, methods, and inheritance.

Objectives:

- Define and instantiate classes.
- Work with attributes and methods.
- Understand inheritance and composition.

Lesson 7.1: Why Learn About Classes?

Reading References

- Zelle: Chapter 4, pages 65–67; Chapter 10, pages 247–248
- Matthes: Chapter 9, page 157

Classes as Blueprints A **class** is a blueprint or template for creating objects. An **object** is a concrete instance created from that blueprint.

Analogy: A "Car" class defines that every car has a `color` and a `speed` (attributes) and can `accelerate()` and `brake()` (methods). Your red sedan and your friend's blue pickup are two separate **objects** of the same "Car" class.

Key OOP Concepts

- **Encapsulation** — Bundling data and methods into a single unit.
- **Abstraction** — Hiding complexity from the outside.
- **Inheritance** — New class reuses functionality of existing class.
- **Polymorphism** — Same method name behaves differently for different classes.

Skills Checklist

☐ I can explain the basic idea of Object-Oriented Programming.
☐ I can describe a class as a blueprint and an object as an instance.

Lesson 7.2: Creating and Using a Class

Reading References

- Zelle: Chapter 10, pages 248–254
- Matthes: Chapter 9, pages 158–162

```
class Dog:
    """A simple attempt to model a dog."""

    def __init__(self, name, age):
        """Initialize name and age attributes."""
        self.name = name
        self.age = age
        print(f"A new dog named {self.name} has been created!")

# Create instances
my_dog = Dog("Buddy", 3)
your_dog = Dog("Lucy", 1)

print(f"My dog's name is {my_dog.name}.")
print(f"Your dog's name is {your_dog.name}.")
```

Class Syntax The `__init__()` method runs automatically when a new object is created. `self` refers to the specific instance being created.

Skills Checklist

☐ I can define a basic Python class.
☐ I can define the `__init__` method within a class.
☐ I understand the purpose of the `self` parameter.
☐ I can create an instance (object) of a class.

Lesson 7.3: Working with Attributes and Methods

Reading References

- Matthes: Chapter 9, pages 162–166

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sit(self):
        """Simulate a dog sitting."""
        print(f"{self.name} is now sitting.")

    def roll_over(self):
        """Simulate a dog rolling over."""
        print(f"{self.name} rolled over!")
```

```
    def get_older(self, years_to_add):
        """Add years to the dog's age."""
        self.age += years_to_add
        print(f"{self.name} is now {self.age} years old.")

my_dog = Dog("Buddy", 3)
my_dog.sit()
my_dog.roll_over()
my_dog.age = 4          # Direct attribute modification
my_dog.get_older(2)     # Method-based modification
```

Calling Methods

Skills Checklist

☐ I can access an object's attributes using dot notation.
☐ I can define and call methods within a class.
☐ I can use a method to change an object's attribute.

Lesson 7.4: Inheritance

Reading References

- Zelle: Chapter 12, pages 351–352
- Matthes: Chapter 9, pages 167–170

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        return f"{self.year} {self.make} {self.model}".title()

    def read_odometer(self):
        print(f"This car has {self.odometer_reading} miles on it.")


class ElectricCar(Car):
    def __init__(self, make, model, year):
        super().__init__(make, model, year)   # Call parent's __init__
        self.battery_size = 75

    def describe_battery(self):
        print(f"This car has a {self.battery_size}-kWh battery.")
```

```
my_ev = ElectricCar('ford', 'mustang mach e', 2023)
print(my_ev.get_descriptive_name())   # Inherited method
my_ev.describe_battery()              # New method
my_ev.read_odometer()                 # Inherited method
```

Creating a Child Class

`super()` Function `super().__init__()` calls the parent class's `__init__()` method, ensuring inherited attributes are properly initialized.

Skills Checklist

☐ I can define a child class that inherits from a parent class.
☐ I can use `super().__init__()` to properly initialize inherited attributes.
☐ I can add new attributes and methods to a child class.
☐ I can override a parent method in a child class.

Lesson 7.5: Instances as Attributes (Composition)

Composition — "Has-A" Relationship Instead of inheriting, a class **contains an instance of another class** as an attribute.

```
class Battery:
    def __init__(self, battery_size=75):
        self.battery_size = battery_size

    def describe_battery(self):
        print(f"This car has a {self.battery_size}-kWh battery.")

    def get_range(self):
        if self.battery_size == 75:
            range_miles = 260
        else:
            range_miles = 310
        print(f"About {range_miles} miles of range.")


class ElectricCar:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.battery = Battery()   # Composition: ElectricCar HAS A Battery


my_ev = ElectricCar('ford', 'mustang mach e', 2023)
```

```
my_ev.battery.describe_battery()
my_ev.battery.get_range()
```

Skills Checklist

☐ I can explain the concept of composition in OOP.
☐ I can create an instance of one class and assign it as an attribute of another.
☐ I can access attributes and methods of a nested object.

Unit 8: Files and Exceptions

Overview

This unit covers interacting with text files for reading and writing data, handling common errors gracefully using exceptions, and storing data using JSON.

Objectives:

- Read from and write to files.
- Handle exceptions with `try-except` blocks.
- Store and load data using JSON.

Lesson 8.1: Reading from a File

Reading References

- Zelle: Chapter 5, pages 125–127
- Matthes: Chapter 10, pages 184–189

```
# Reading entire file
with open('my_notes.txt', 'r') as file_object:
    contents = file_object.read()
print(contents)

# Reading line by line
with open('my_notes.txt', 'r') as file_object:
    for line in file_object:
        print(line.strip())

# Reading all lines into a list
with open('my_notes.txt', 'r') as file_object:
    lines = file_object.readlines()
```

Reading Files The `with` statement **automatically closes the file** even if an error occurs.

File Modes

Mode	Description
`'r'`	Read (default)
`'w'`	Write (creates new or overwrites)
`'a'`	Append (adds to end)

Skills Checklist

☐ I can open a file for reading using `with open()`.
☐ I can read the entire content of a file into a string.
☐ I can read a file line by line using a `for` loop.
☐ I can remove whitespace/newlines using `.strip()`.

Lesson 8.2: Writing to a File

Reading References

- Zelle: Chapter 5, pages 127–128
- Matthes: Chapter 10, pages 190–192

```
# Write mode (overwrites existing content)
with open('programming_message.txt', 'w') as f:
    f.write("I love programming in Python!\n")

# Append mode (adds to end)
with open('programming_message.txt', 'a') as f:
    f.write("It's a very versatile language.\n")
    f.write("I'm excited to learn more!\n")
```

Important: The `.write()` method does not add newlines automatically. Use `\n`.

Skills Checklist

☐ I can open a file for writing using `"w"` mode.
☐ I can open a file for appending using `"a"` mode.
☐ I can write a string to a file.
☐ I can add newlines to a file using `\n`.

Lesson 8.3: Exceptions — Handling Errors

Reading References

- Zelle: Chapter 7, pages 177–179
- Matthes: Chapter 10, pages 192–199

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
print("Program continues after error handling.")
```

`try-except` Block

Common Exception Types

Exception	When it occurs
`ValueError`	Invalid value (e.g., `int("abc")`)
`ZeroDivisionError`	Division by zero
`FileNotFoundError`	File doesn't exist
`IndexError`	List index out of range
`KeyError`	Dictionary key doesn't exist
`TypeError`	Wrong type (e.g., `"5" + 5`)

Examples **Example 8.3.1: Handling ZeroDivisionError**

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Example 8.3.2: Handling ValueError

```
try:
    num = int(input("Enter a number: "))
    print(f"You entered: {num}")
except ValueError:
    print("Error: That was not a valid number.")
```

Skills Checklist

☐ I can use a `try-except` block to handle errors.
☐ I can name common exception types and what causes them.
☐ I can catch specific exception types.

Lesson 8.4: The `else` and `finally` Blocks

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("That's not a number.")
else:
    print(f"You entered {num}.")   # Only runs if no exception
```

`else` Block — Runs only on success

```
file_handle = None
try:
    file_handle = open('data.txt', 'w')
    file_handle.write("Some data.")
except TypeError:
    print("A type error occurred.")
finally:
    if file_handle:
        file_handle.close()     # Always closes the file
        print("File closed.")
```

`finally` Block — Always runs (cleanup)

Skills Checklist

☐ I can use an `else` block for code that runs only on success.
☐ I can use a `finally` block for code that always runs.

Lesson 8.5: Storing Data with JSON

Reading References

- Matthes: Chapter 10, pages 200–204

The `json` Module JSON (JavaScript Object Notation) is a lightweight format for storing and exchanging data. Python's `json` module converts between Python objects and JSON.

```
import json

# Save data
user_settings = {
    "username": "coder_kid",
    "theme": "dark",
    "notifications_on": True,
    "favorite_numbers": [7, 13, 42]
```

```
}

with open('user_settings.json', 'w') as f:
    json.dump(user_settings, f, indent=4)

# Load data
try:
    with open('user_settings.json', 'r') as f:
        loaded_settings = json.load(f)
    print(f"Username: {loaded_settings['username']}")
    print(f"Theme: {loaded_settings['theme']}")
except FileNotFoundError:
    print("Settings file not found.")
except json.JSONDecodeError:
    print("Error: Could not decode settings file.")
```

JSON<->Python Mapping

JSON	Python
Object `{}`	Dictionary `{}`
Array `[]`	List `[]`
String	`str`
Number	`int` or `float`
Boolean	`bool`
null	`None`

Skills Checklist

☐ I can explain what JSON is and why it's used.
☐ I can use `json.dump()` to save Python data to a JSON file.
☐ I can use `json.load()` to load JSON data from a file.
☐ I can save and load lists of dictionaries using JSON.

Unit 9: Testing and Data Analysis

Overview

This unit introduces testing code for correctness and explores basic text-based data analysis.

Lesson 9.1: Testing a Function

Reading References

- Matthes: Chapter 11, pages 209–217

```python
# my_functions.py
def add_two_numbers(a, b):
    return a + b
```

```python
# test_my_functions.py
import unittest
from my_functions import add_two_numbers

class TestAddNumbers(unittest.TestCase):
    def test_positive_numbers(self):
        result = add_two_numbers(5, 3)
        self.assertEqual(result, 8)

    def test_negative_numbers(self):
        result = add_two_numbers(-5, -3)
        self.assertEqual(result, -8)

    def test_mixed_numbers(self):
        result = add_two_numbers(10, -4)
        self.assertEqual(result, 6)

if __name__ == '__main__':
    unittest.main()
```

Unit Testing with `unittest`

Common Assertion Methods

Method	Description
`assertEqual(a, b)`	Check a == b
`assertTrue(x)`	Check x is True
`assertFalse(x)`	Check x is False
`assertIn(a, b)`	Check a in b
`assertIsNone(x)`	Check x is None

Skills Checklist

☐ I can create a basic test class using `unittest.TestCase`.
☐ I can define test methods within a test class.
☐ I can use `assertEqual()` to check for expected results.
☐ I can run unit tests from the terminal.

Lesson 9.2: Testing a Class

The `setUp()` Method `setUp()` runs **before every test method**, providing a fresh instance each time:

```
class TestEmployee(unittest.TestCase):
    def setUp(self):
        """Create an employee instance for all test methods."""
        self.employee = Employee('janis', 'joplin', 60000)

    def test_give_default_raise(self):
        self.employee.give_raise()
        self.assertEqual(self.employee.salary, 65000)

    def test_give_custom_raise(self):
        self.employee.give_raise(10000)
        self.assertEqual(self.employee.salary, 70000)
```

Skills Checklist

☐ I can write tests for a Python class.
☐ I can use `setUp()` to prepare my tests.

Lesson 9.3: Introduction to Random Numbers

```
import random

# Random integer (inclusive)
die_roll = random.randint(1, 6)
print(f"You rolled: {die_roll}")

# Random float between 0.0 and 1.0
rand_float = random.random()

# Random choice from list
colors = ["red", "blue", "green"]
chosen = random.choice(colors)
print(f"Chosen color: {chosen}")
```

Skills Checklist

☐ I can import the `random` module.
☐ I can generate a random integer within a range.
☐ I can select a random item from a list.

Lesson 9.4: Analyzing Text Data

```
sentence = "the quick brown fox jumps over the lazy dog the quick brown fox"
words = sentence.lower().split()

word_frequencies = {}
for word in words:
    if word in word_frequencies:
        word_frequencies[word] += 1
    else:
        word_frequencies[word] = 1

print("Word Frequencies:")
for word, count in word_frequencies.items():
    print(f"'{word}': {count}")
```

Skills Checklist

☐ I can read text content from a file.
☐ I can split a string of text into individual words.
☐ I can use a dictionary to count the frequency of words.

Lesson 9.5: Working with CSV Data

```
import csv

filename = 'students.csv'
try:
    with open(filename, 'r') as file_object:
        reader = csv.reader(file_object)
        header_row = next(reader)
        print(f"Columns: {header_row}")
        for row in reader:
            print(row)
except FileNotFoundError:
    print(f"Error: The file {filename} was not found.")
```

```
with open('students.csv', 'r') as f:
    header = f.readline().strip()
    for line in f:
        data = line.strip().split(',')
        name = data[0]
        age = data[1]
        grade = data[2]
        print(f"Name: {name}, Age: {age}, Grade: {grade}")
```

Manual CSV Parsing

Skills Checklist

☐ I can explain what CSV data is.
☐ I can read a CSV file line by line.
☐ I can split a line of CSV data into individual values.

Unit 10: Project-Based Learning — Building Text-Based Applications

Overview

This unit guides students through designing and building larger, interactive text-based applications, integrating all previously learned concepts.

Objectives:

- Practice modular programming and code organization.
- Develop a text-based interactive program.
- Apply top-down design principles.

Lesson 10.1: Designing Your Project

Reading References

- Zelle: Chapter 9, pages 225–235
- Matthes: Part II: Projects, pages 225–226

Top-Down Design **Top-down design** (stepwise refinement) breaks a complex problem into smaller, more manageable subproblems.

The Design Process:

1. Analyze the Problem

- Identify **inputs** (information the program needs).
- Identify **outputs** (results it must produce).
- Define the scope of the project.

2. Determine the Major Components Write a high-level algorithm in plain English:

```
show_greeting()
get_user_input()
calculate_cost()
print_ticket()
```

3. Refine Each Component Repeat the design process for each component, adding detail until simple enough to code.

Benefits of Top-Down Design

- **Reduces Complexity** — Focus on one piece at a time.
- **Encourages Reusability** — Functions written for one part may be useful elsewhere.
- **Simplifies Debugging** — Test each component in isolation.
- **Improves Collaboration** — Clear plan for assigning work.

Accessibility Considerations for I/O Design

- Provide **clear, descriptive prompts** for all input.
- Provide **explicit, labeled output** — avoid relying on visual formatting.
- Print **confirmation messages** after every action.
- Use consistent **output separators** like `---` to group information.

Skills Checklist

☐ I can break down a larger problem into smaller, manageable parts.
☐ I can identify the key features required for a project.
☐ I can plan which data structures will be most suitable for my project's data.
☐ I can outline the main steps or flow of my program.

Lesson 10.2: Building a Text-Based Adventure Game (Part 1)

Reading References

- Matthes: Chapter 6, pages 105–111

```
room_descriptions = {
    "start_room": {
        "description": "You are in a dimly lit cave. There's a narrow passage to the NORTH.",
        "exits": {"north": "tunnel"}
    },
    "tunnel": {
        "description": "The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to a clearing.",
        "exits": {"south": "start_room", "east": "clearing"}
    },
    "clearing": {
        "description": "You've reached a small, sunny clearing. A faint path leads WEST.",
        "exits": {"west": "tunnel"}
    }
}

current_room = "start_room"

while True:
    print("---")
    print(room_descriptions[current_room]["description"])
    command = input("What do you want to do? ").lower()

    if command in room_descriptions[current_room]["exits"]:
        current_room = room_descriptions[current_room]["exits"][command]
```

```
    elif command == "quit":
        print("Exiting game. Goodbye!")
        break
    else:
        print("I don't understand that command or you can't go that way.")
```

Room Definition and Movement

Skills Checklist

- ☐ I can design a way to represent different locations in a text-based game.
- ☐ I can implement player movement based on user commands.
- ☐ I can update and track the player's current location.
- ☐ I can parse simple commands from user input.

Lesson 10.3: Building a Text-Based Adventure Game (Part 2)

```
rooms = {
    "start": {
        "description": "A dark cave. There's a SHINY KEY on the ground.",
        "exits": {"north": "locked_door_room"},
        "items": ["shiny key"]
    },
    "locked_door_room": {
        "description": "A small chamber with a heavy locked door to the NORTH.",
        "exits": {"south": "start"},
        "items": []
    }
}

current_room = "start"
player_inventory = []
door_locked = True

while True:
    print("\n" + rooms[current_room]["description"])
    if rooms[current_room]["items"]:
        print(f"You see: {', '.join(rooms[current_room]['items'])}")
    print(f"Your inventory: {', '.join(player_inventory) if player_inventory else 'empty'}")

    command = input("What do you want to do? ").lower()

    if command == "take shiny key" and "shiny key" in rooms[current_room]["items"]:
        player_inventory.append("shiny key")
        rooms[current_room]["items"].remove("shiny key")
        print("You picked up the shiny key.")
```

```
    elif command == "use shiny key" and current_room == "locked_door_room" and "shiny key" in player_inventory and door_loc
        door_locked = False
        print("You unlocked the door!")
        rooms["locked_door_room"]["exits"]["north"] = "treasure_room"
    elif command in rooms[current_room]["exits"]:
        if command == "north" and current_room == "locked_door_room" and door_locked:
            print("The door is locked! You need a key.")
        else:
            current_room = rooms[current_room]["exits"][command]
    elif command == "quit":
        break
    else:
        print("I don't understand that command.")
```

Adding Items, Inventory, and Puzzles

Skills Checklist

☐ I can add items to locations and manage a player's inventory.
☐ I can implement commands for picking up and dropping items.
☐ I can design and implement a simple puzzle.
☐ I can define win/lose conditions for the game.

Lesson 10.4: Developing a Command-Line Tool

```
import json

TODO_FILE = 'todos.json'
todos = []


def load_todos():
    global todos
    try:
        with open(TODO_FILE, 'r') as f:
            todos = json.load(f)
        print("Tasks loaded.")
    except FileNotFoundError:
        print("No existing tasks found.")
        todos = []


def save_todos():
    with open(TODO_FILE, 'w') as f:
        json.dump(todos, f, indent=4)
    print("Tasks saved.")
```

```
def add_task(task_description):
    todos.append({"description": task_description, "completed": False})
    print(f"Task '{task_description}' added.")


def view_tasks():
    if not todos:
        print("No tasks in your list.")
        return
    print("--- Your To-Do List ---")
    for i, task in enumerate(todos):
        status = "[X]" if task["completed"] else "[ ]"
        print(f"{i + 1}. {status} {task['description']}")
    print("-------------------------")


def mark_completed(task_number):
    try:
        index = int(task_number) - 1
        if 0 <= index < len(todos):
            todos[index]["completed"] = True
            print(f"Task '{todos[index]['description']}' marked as complete.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")


def main_menu():
    print("\n--- To-Do List Menu ---")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Mark Task Complete")
    print("4. Save Tasks")
    print("5. Load Tasks")
    print("6. Quit")
    print("-------------------------")


load_todos()

while True:
    main_menu()
    choice = input("Enter your choice (1-6): ")

    if choice == '1':
        desc = input("Enter task description: ")
        add_task(desc)
    elif choice == '2':
        view_tasks()
```

```
    elif choice == '3':
        view_tasks()
        task_num = input("Enter task number to mark complete: ")
        mark_completed(task_num)
    elif choice == '4':
        save_todos()
    elif choice == '5':
        load_todos()
    elif choice == '6':
        save_todos()
        print("Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

Simple To-Do List Tool

Skills Checklist

☐ I can design an interactive menu system for a tool.
☐ I can use a `while` loop to maintain continuous user interaction.
☐ I can break down the tool's functionality into separate functions.
☐ I can implement file I/O to save and load data.

Lesson 10.5: Refinement and User Experience

Principles for Accessible Text Programs

1. **Clear prompts** — Tell the user exactly what to enter and in what format.
2. **Consistent output** — Use the same format for similar information.
3. **Graceful error handling** — Never let the program crash on bad input.
4. **Helpful error messages** — Tell the user what went wrong and how to fix it.
5. **"Help" commands** — Provide a way to list available commands.
6. **Confirmation messages** — Confirm every action taken.

```
def get_positive_number(prompt):
    """Prompts user until a positive integer is entered."""
    while True:
        try:
            num = int(input(prompt))
            if num > 0:
                return num
```

```
            else:
                print("Please enter a positive number.")
        except ValueError:
            print("Invalid input. Please enter a whole number.")


age = get_positive_number("Enter your age: ")
print(f"You entered age: {age}")
```

Robust Input Validation

Skills Checklist

☐ I can write clear, descriptive prompts for user input.
☐ I can handle invalid input gracefully using `try-except`.
☐ I can provide helpful error messages.

Unit 11: Capstone Project — Accessible Personal Data Manager

Overview

Students will develop a fully functional, screen-reader-accessible graphical application using NiceGUI that allows users to manage a collection of structured personal data.

Project Goal: Build a simple application to manage a list of structured items.

Project Options:

- Contact Manager: name, phone, email, notes
- Book/Movie Tracker: title, author/director, genre, status
- Simple To-Do List: task description, due date, status
- Simple Inventory: item name, quantity, location, notes

Core Functionality Requirements

1. **Add New Item** — A form to input details for a new item.
2. **View All Items** — Display items in a clear, accessible list format.
3. **Edit Item** — Select an existing item and modify its details.
4. **Delete Item** — Remove an item from the collection.
5. **Save/Load Data** — Persist the data to a JSON file.

Setup

```
pip install nicegui
```

Guided Implementation

Lesson 11.1: Introduction to NiceGUI **Task 1:** Run the `app_skeleton.py` file and navigate using the keyboard and screen reader to understand the existing structure.

Task 2: Define your data model class:

```
class Contact:
    def __init__(self, name, phone, email, notes):
        self.name = name
        self.phone = phone
        self.email = email
        self.notes = notes
```

Task 3: Implement data persistence using the `json` module within the `load_data()` and `save_data()` functions.

Lesson 11.2: Data Model and Storage **Task 1:** Complete the `add_item_logic()` function:

1. Get values from input fields using `.value` attribute.
2. Validate (e.g., name cannot be empty).
3. Provide accessible feedback: `ui.notify("Name is required!", type='negative')`.

Task 2: Create a new item object and add it to `all_items`.

Task 3: Clear input fields and provide success notification.

Lesson 11.3–11.5: Displaying and Managing Items **Implement `update_item_display()`:**

1. Clear the display: `items_display_column.clear()`.
2. Loop through `all_items` and create `ui.card` for each.
3. Update `item_selection_dropdown.options`.

Implement `update_item_logic()`:

1. Find the item in `all_items`.
2. Update attributes from edit form fields.
3. Call `update_item_display()` and `save_data()`.

Implement `delete_selected_item_logic()`:

1. Find and remove the item from `all_items`.
2. Call `update_item_display()`.
3. Provide accessible feedback.

Assessment Criteria

Functionality (60%)

☐ Ability to add new items with all specified attributes.
☐ Correct display of all current items.
☐ Correct editing of existing items.
☐ Correct deletion of items.
☐ Accurate saving and loading to/from `data.json`.
☐ No major bugs or crashes during typical usage.

Code Quality & Structure (20%)

☐ Clear and concise Python code.
☐ Appropriate use of functions and/or classes.
☐ Meaningful variable and function names.
☐ Adequate comments where complex logic exists.
☐ Proper indentation.

Accessibility & User Experience (15%)

☐ All interactive elements are correctly labeled for screen readers.
☐ Clear and consistent user feedback via `ui.notify()`.
☐ Logical flow and presentation of information.
☐ No visual-only cues for critical information.

README and Reflection (5%)

☐ Complete and clear README.
☐ Thoughtful reflection on development and accessibility challenges.

Deliverables

1. `app.py` — Complete NiceGUI application code.
2. `data.json` — Example JSON file demonstrating saved data.
3. `README.md` — Including:

   - Project title and student name.
   - Description of the application.
   - Instructions on how to run: `python app.py`.
   - Instructions for interacting with a screen reader.
   - Reflection on challenges faced.

Supplemental: AI Tools and Version Control

AI-Powered Coding Tools **Benefits:**

- Increased productivity and speed.
- Syntax and idiom discovery.
- Rapid prototyping.

Critical Warning: AI tools can produce code that looks correct but contains subtle bugs. **Always:**

1. Read every line of AI-generated code.
2. Verify the logic and intent.
3. Look for familiar error types (indentation, name errors, type errors).
4. Run and test incrementally.
5. Add your own comments and docstrings.

The Golden Rule: You are responsible for your code. Treat AI suggestions as a first draft that needs meticulous editing.

Git and Version Control **Git** is the most widely used distributed version control system, invented in 2005 by Linus Torvalds for the Linux kernel project.

Core workflow:

1. **Working directory** — Make changes to files.
2. **Staging area** — `git add` — Stage changes.
3. **Repository** — `git commit` — Save a permanent snapshot.
4. **Remote** — `git push` / `git pull` — Sync with GitHub.

Why Git is especially valuable with AI code:

- **Rollback** — If AI-generated code breaks your app, revert instantly.
- **`git blame`** — See exactly which commit introduced a bug.
- **Branches** — Test AI experiments in isolation without affecting main code.
- **Pull requests** — Review AI-generated code before merging.

Basic commands:

```
git init                    # Initialize a repository
git add .                   # Stage all changes
git commit -m "Add feature" # Save a snapshot
git log                     # View history
git diff                   # See what changed
git checkout -- file.py     # Discard changes to a file
```